GHATS

USER'S MANUAL

Version 4.0.0 - 29 August 2026

General
High-Energy
Aperiodic
Timing
Software

Author: Tomaso M. Belloni (INAF - Osservatorio Astronomico di Brera)

From 26 August 2023, and after the passing of Tomaso, development has been taken over by:

Mariano Méndez (Kapteyn Astronomical Institute, University of Groningen)
Federico García (Instituto Argentino de Radioastronomía)
Sara Motta (INAF - Osservatorio Astronomico di Brera)

NOISE, n. A stench in the ear. Undomesticated music. The chief product and authenticating sign of civilization.

(A. Bierce, "The Devil's Dictionary")

It is of a flawless white color, right from its nose to its magnificent tail.

(Mahabharata, The Story of Garuda)

Revision History

This section summarizes the main user-visible changes. The version 4.0.0 entry is extracted from `CHANGELOG.md` and `RELEASES/v4.0.0.md`. Older entries below are retained from the legacy manual.

Version 4.0.0 – 29 August 2026

Version 4.0.0 is a major feature release and the largest functional expansion of GHATS since version 3.3.0.

- Added a new `BISPECTRUM` module for higher-order Fourier timing analysis, including one- and two-dimensional bispectrum calculations, bicoherence and normalized bispectral products, hypotenuse-domain analysis, cross-bispectra, two-dimensional rebinning and plotting, and FITS input/output for bispectrum products.
- Added support for Einstein Probe and IXPE, including mission-specific event reading, channel selection, time-resolution detection, and FITS metadata handling.
- Updated support for RXTE, AstroSat CZTI and LAXPC, Insight-HXMT HE/ME/LE, NICER, NuSTAR, Swift, and XMM-Newton, with more consistent FFT generation and metadata handling across missions.
- Added and expanded analysis utilities, including PDS metafiles in `ghx`, FFT generation from ASCII light curves, FFT/PDS splitting utilities, PDS concatenation utilities, Poisson-level estimation over selected frequency intervals, and improved `gh_info` support for both PDS and FFT products.
- Updated `ghats_all` to produce quick-look power spectra directly from GHATS FFT files and to warn about malformed legacy products.
- Added a central help system with alphabetical and task-oriented indices, command-line help support, and expanded `/HELP` text for many user-facing routines.
- Added this LaTeX-based User Manual, including the native file-format reference in Appendix B.
- Standardized FFT-length handling so that the GHATS `NFT` header field is written as an IDL `LONG`, preserving the intended binary header layout.
- Improved validation and reporting of GHATS FFT/PDS headers, including diagnostics for products produced by older command-line paths that wrote `NFT` with the wrong integer size.
- Improved averaging, rebinning, plotting, dynamic-PDS, light-curve, and housekeeping routines, together with XSPEC-export and Poisson-noise handling.
- Fixed malformed FFT/PDS headers caused by writing the FFT length as a 16-bit IDL integer instead of the required 32-bit IDL `LONG`.

- Fixed FFT quick-look processing so powers are calculated from individual FFT records rather than from averaged complex amplitudes, and added safeguards for quick-look plots with no positive powers.
- Fixed an obsolete AstroSat/CZTI branch that prevented the distributed default configuration from compiling.

> **Note.** The intended GHATS binary-header layout has not changed. Correct products from earlier versions are expected to remain compatible. Products affected by the older malformed `NFT` field should be regenerated before scientific analysis.

Version 1.0.1

- Fixed uppercase/lowercase for POWER/FFT in `gh_xte`.
- Fixed crash in `GH_CROSS`.
- Added `GH_PLOT_LAG`.
- Added `GH_PLOT_COH`.
- Fixed wrong stretch time in `GHATS_ALL` output.
- Fixed wrong stretch time in `GH_XTE` output to terminal and log file.

Version 1.0.2

- Added `GH_CROSS_RANGE`.
- Removed `GH_PLOT_CROSS`, a duplicate of `GH_PLOT_LAG`.
- Fixed `GH_XTE` sliding-mode bug in turbo mode.
- Fixed `GH_XTE` recalculation of the number of PDS in non-turbo mode.
- Fixed `GH_XTE` housekeeping-file discovery in turbo mode when data are elsewhere.
- Removed calls to `FSC_FILE_BASENAME`, and removed that routine from the package for compatibility.
- Restricted `GH_XTE` channel ranges to available channels rather than assuming the input range was available.
- Made `GH_CROSS_RANGE` intercept the case in which zero or one bins are to be averaged.
- Changed the `GH_CROSS` and `GH_CROSS_RANGE` `/TIMELAG` keyword to `/TLAG` for uniqueness.
- Fixed time-selection bugs in `GH_CROSS` and `GH_CROSS_RANGE`.
- Changed the default frequency rebinning in `GH_DYN` from `-100` to `1`, i.e. no rebinning.
- Added `GH_COLORS`.
- Fixed timing bugs in `GHX`, `GH_DYN`, `GH_COLORS`, `GH_HK`, `GH_CROSS`, and `GH_CROSS_RANGE`.

Version 1.1.0

- Added the windowing option to GH_XTE.
- Added GH_SWIFT.
- Renamed GH_HK to GH_HK_XTE.
- Renamed GH_PLOT_HK to GH_PLOT_HK_XTE.
- Increased the maximum number of input files in GH_XTE to 1000.
- Fixed empty-data-stretch bugs in GH_XTE and GH_SWIFT.
- Fixed the GH_CROSS Poisson estimate, which had been valid only for RXTE.

Version 1.1.1

- Added output of the average complex cross-spectrum value to GH_CROSS_RANGE.
- Added GH_XMM.
- Added GH_NUSTAR.

Version 2.0.0

- Fixed the GH_CROSS_RANGE time-lag output.
- Fixed bugs that limited time rebinning in GH_XTE, GH_SWIFT, GH_XMM, and GH_NUSTAR.
- Fixed logical-unit release in GH_CROSS.
- Fixed the GH_CROSS Poisson estimate for instruments different from PCA.
- Fixed logical-unit release in GH_CROSS_RANGE.
- Increased the maximum number of GTI intervals in GH_NUSTAR.
- Fixed a bug in GH_PLOT_HK_XTE.
- Fixed a file-logical-unit bug in GHX.
- Fixed command-line input bugs in GH_XTE, GH_SWIFT, GH_XMM, and GH_NUSTAR.
- Added GH_LAXPC.
- Added GH_CZTI.
- Fixed a GH_CROSS_RANGE time-lag-computation bug.
- Changed GHX indices for the /SEL option to start from zero, for IDL WHERE compatibility.

Version 3.2.0

- Added the /RATE keyword to GH_CROSS.
- Added an absolute-value check on the time difference used by GH_CROSS to test file compatibility.
- Added /POIVALUE to GH_CROSS for the real part of the Poisson contribution to be subtracted.

- Added a keyword to `GH_CROSS` for cross-spectrum output.
- Added reading of the `TIMEZERO` keyword from GTI files and extensions in the `GH_*` production routines.

Version 3.3.0

- First release after the passing of Tomaso Belloni.
- Added `GH_CROSS_RE_IM_NEW`.
- `GH_CROSS_RE_IM_NEW`: added real and imaginary cross-spectrum output.
- Added the option to rotate the cross spectrum by 45 degrees in `GH_CROSS_RE_IM_NEW`.
- Added `GH_CS`.
- Made `GH_CS` use `GH_CROSS_RE_IM_NEW` twice; in the process it corrects for deadtime-induced cross talk and, if necessary, for partial correlation when the subject and reference bands overlap.
- Added to `GH_CS` the option to compute coherence errors in the case of high powers and low coherence, following equation 9 of Vaughan and Nowak (1997), using `GH_LOWCOH_SSQ` and `GH_LOWCOH_SSQ_CI`.
- Added `CONCATENATE_PDS`.
- Added PDS-file concatenation in `CONCATENATE_PDS`.
- Added `CONCATENATE_FFT`.
- Added FFT-file concatenation in `CONCATENATE_FFT`.
- Added `GH_DYN_FITS`, which saves and plots a time-frequency image from `GH_DYN` to a FITS file.
- Added the Python GUI tool `GH_DYN_FITS_TK.py`, which plots spectrograms created with `GH_DYN_FITS`.

Contents

Chapter 1

Introduction

GHATS is the General High-Energy Aperiodic Timing Software package. It is a set of IDL/GDL routines for the production and analysis of high-energy timing products, with particular emphasis on aperiodic variability and Fourier timing. The first part of this introduction preserves the motivation and history of the original manual written by Tomaso M. Belloni. The final section records the new start of the project after release 3.2.0 and the beginning of the 3.3.0 line.

1.1 The Need for a Timing Package

The long-living RossiXTE mission was aimed at observing time-variable phenomena from X-ray sources. Its main instrument, a large proportional counter with large collecting area, high time resolution, good calibration, and well understood dead-time effects, was designed for the study of fast variability.

The XRONOS package distributed by NASA/HEASARC provides tools for the timing analysis of periodic phenomena such as pulsars. For aperiodic timing analysis of RXTE data, however, the community did not have a comprehensive and efficient package. Each group was therefore left to its own devices and had to develop a custom set of tools.

1.2 The Background Behind GHATS

At INAF-OAB, Tomaso Belloni started writing a package devoted to RXTE/PCA data. The HEXTE instrument, although called the High Energy X-ray Timing Experiment, is almost unusable for the kind of timing analysis for which this software was developed. The philosophy behind the package can be summarized in two points.

First, the program had to work directly from RXTE raw files, without forcing the user to produce light curves before the analysis. In other words, the program itself would accumulate its own light curves, which would be discarded at the end of the production step.

Second, Fourier timing analysis is naturally divided into two separate tasks. The first is the production of Power Density Spectra (PDS) or Fast Fourier Transforms (FFT). This is an almost non-interactive process: the user supplies the input parameters and the program delivers the final product. Some interaction remains necessary because the possible parameter ranges depend on the specific data type and can only be known after reading the input files. The second task is the analysis of the products: reading PDS or FFT files, manipulating them, plotting them, fitting them, and comparing different timing quantities. This second task is highly interactive.

Given these starting points, the best solution was to sacrifice some speed and write the software in an integrated graphical environment. Although IDL has its shortcomings,

it was a widely used package in high-energy astrophysics and provided access to useful existing libraries. Its main disadvantage is that it is commercial software. The GNU Data Language (GDL), a public-domain clone of IDL, has therefore always been important for the accessibility of the package. Although GDL was not complete when the original manual was written, it was already sufficiently advanced for many GHATS purposes.

1.3 A New Package with Limited Distribution

From this work, a package called MU was born. Initially used only within the institute, it was later shared with selected users in Groningen, Rome, Mumbai, and Kolkata. MU worked only on RXTE/PCA data and continued to do so. The upcoming AstroSat mission provided the incentive to extend the software and adapt it to other instruments.

Although the package itself was in good shape, the data format was too RXTE-specific and needed to be changed into something more general. This is how GHATS was born. Release 1.0.0 had only been used by Tomaso for testing when it was first released. Users of MU recognized essentially the same software, with command names changed. The changes in data format were mostly invisible to the user, while the extensions to other missions were added to the existing code base.

1.4 The Future

The original manual closed by saying that GHATS needed testing by a larger number of users. In particular, the GDL version had been used only by very few people and would certainly benefit from broader use. That remains part of the spirit of the project: the software is meant to be used, tested, corrected, and extended by the timing community, while preserving the definitions and file formats that make its products mutually compatible.

1.5 A New Start

Sara Motta, Federico Garcia, and Mariano Mendez used GHATS for several years and discussed many possible improvements, bugs, and extensions with Tomaso. They also contributed parts of some routines. The last version of the code officially received from Tomaso was 3.2.0, from February 2023, about six months before he passed away.

Since then, the Groningen copy of the code received a few small changes to existing routines and a number of new routines for studying the cross spectrum and the coherence function. As a legacy to Tomaso's work, version 3.2.0 was uploaded to GitHub, and version 3.3.0 was started from it. This is the line intended for further development.

Besides adding those routines, the manual and the links to the necessary libraries were updated, instructions for installing and running GHATS in GDL were included, a `PYTHON` directory was added for Python-based GHATS tools, and a new walkthrough for the cross spectrum was added.

We miss Tomaso, and his enthusiasm for adding things to the code whenever new ideas came up. We would have very much liked to share this new step with him. In the meantime, we try to honour his memory by doing good work with this software. We hope that new users will do the same.

1.6 This Edition

This LaTeX edition keeps the structure and spirit of the previous manual while making the text easier to update as GHATS evolves. Later chapters describe how to produce PDS

and FFT products, read and average them, make the standard plots, use the cross-spectral routines, create dynamic PDS products, and export some results for XSPEC. The appendix records the internal GHATS file formats so that future tools can read and write compatible products.

Chapter 2

Installation and Dependencies

2.1 The Package

Originally, the GHATS package could be obtained from INAF-OAB and was not distributed broadly. This was because Tomaso did not have the resources to support the software for a large community. From version 3.3.0, the code is available from the GitHub repository so that the community can use it, test it, and contribute fixes or contributed routines on a development branch.

2.2 Unpacking

You can unpack or clone the package in your preferred directory. The IDL/GDL part of GHATS is distributed as source code, so the tree is made of ASCII routines rather than compiled binaries. A typical checkout contains the core directories

- `FFT`: routines used in FFT and PDS production;
- `ANALYSIS`: routines for interactive analysis of GHATS products;
- `CROSS_ANALYSIS`: routines dedicated to cross spectra, lags, and coherence;
- `CONTRIBUTED`: contributed routines;
- `PYTHON`: Python-based GHATS tools, including the command-line helper directory;
- `DOCS`: documentation and manual sources.

The tree also contains mission-specific directories, for example `NICER`, `NuSTAR`, `Swift`, `XMM`, `IXPE`, `ASTROSAT`, `Huiyan`, and `Einstein_Probe`. These contain the routines that know how to read the native event products of the corresponding instruments and produce GHATS-format PDS or FFT files.

If your local installation includes launcher scripts such as `ghats` for IDL or `gghats` for GDL, put the one you use in a directory that is in your shell `PATH`, and make it executable. Otherwise, the same setup can be done directly in your shell by defining the environment variables described below and then starting IDL or GDL.

2.3 Software Dependencies

You need the following software and libraries installed:

1. An updated and running version of IDL or GDL. If you use a precompiled GDL for your architecture, you may still need the source distribution, because many procedures are external. GDL can be installed following the instructions at https://github.com/gnudatalanguage/gdl/wiki/GDL-on-Linux.
2. An updated version of the IDL Astronomy User's Library: https://asd.gsfc.nasa.gov/archive/idlastro/.
3. An updated version of Craig Markwardt's IDL library: https://pages.physics.wisc.edu/~craigm/idl/idl.html.

Not every routine in those libraries is required by every GHATS command, but they provide many utilities that the package expects to have available.

2.4 Environment Setup

To run GHATS, your shell must know where the repository is, where IDL or GDL is installed, and where the required IDL libraries are. The key variables are `MU_PATH`, `IDL_STARTUP`, `IDL_DIR`, and `IDL_PATH`.

For a Bourne-like shell such as `bash`, a typical IDL setup is

```
export MU_PATH=/path/to/GHATS
export IDL_STARTUP=$MU_PATH/ANALYSIS/ghats.pro
export IDL_DIR=/path/to/idl
export IDL_PATH=+$MU_PATH:+/path/to/astron.dir:+/path/to/markwardt:+$IDL_DIR/lib
export PATH=$PATH:$IDL_DIR/bin
export PATH=$PATH:$MU_PATH/PYTHON
```

Replace the paths with the locations on your machine. The leading `+` in `IDL_PATH` tells IDL/GDL to search a directory and its subdirectories.

For a C-shell-like shell such as `tcsh`, the same setup uses `setenv`:

```
setenv MU_PATH /path/to/GHATS
setenv IDL_STARTUP $MU_PATH/ANALYSIS/ghats.pro
setenv IDL_DIR /path/to/idl
setenv IDL_PATH +$MU_PATH:+/path/to/astron.dir:+/path/to/markwardt:+$IDL_DIR/lib
setenv PATH ${PATH}:$IDL_DIR/bin:$MU_PATH/PYTHON
```

These commands can be put in your shell startup file if you want them to be available in every terminal session. For example, `bash` users typically use `.bashrc` or `.bash_profile`, while `tcsh` users commonly use `.login`.

2.5 Running the Program

After the environment is set, start the appropriate interpreter or launcher. If you use a launcher script, the commands are typically

```
$ ghats
```

for IDL, or

```
$ gghats
```

for GDL. If you start IDL directly, the startup file should load the GHATS environment and change the prompt to

```
Ghats>
```

At that point the GHATS routines can be called directly. The following chapters give examples of producing PDS and FFT files, reading them back, plotting timing products, and using the cross-spectral tools.

Chapter 3

Quick Start

This chapter gives a first complete GHATS workflow. The example follows the RXTE/PCA walkthrough in the legacy manual: prepare an input list, produce a PDS file, inspect it, extract a light curve, average the PDS, rebin it, and plot it.

3.1 Input Data

For RXTE/PCA, the starting point is normally science-array or event data. The supported data modes include binned data, single-bit data, event data, and preprocessed GoodXenon data. The time and channel resolution depend on the specific data mode.

The input to the PDS/FFT production program can be:

- a single science file;
- a metafile, identified by a leading `@`, containing one input filename per line;
- a meta-metafile, identified by a leading `@@`, containing one metafile name per line.

Files listed in the same metafile must be homogeneous: same mode, same time resolution, same energy ranges, and same binning. For RXTE/PCA data, time sorting is not needed by the production routine.

```
$ ls FS3f* > @sb.lis
$ ls FS3b* > @event.lis
$ ls @sb.lis @event.lis > @@all.lis
```

3.2 Producing a PDS

Before running the production routine, compressed RXTE files and the required auxiliary files should be uncompressed.

```
$ gzip -d FS4a* FH5a*
```

Start GHATS and run the production routine:

```
Ghats> gh_xte
```

When called without arguments, `gh_xte` runs interactively. A file selection window appears; in this example the selected file is `@event.lis`. The program then shows the available channel ranges and asks whether to use all of them or a selection.

> **Note.** If you are running IDL, `gh_xte` can use the `/TURBO` keyword, which switches to a faster FFT engine from IDL_EXTRACT by C. Markwardt.

The routine asks whether to produce a PDS, stored as a power-spectrum file, or an FFT, stored as a file of Fourier transforms. Unless the analysis requires time lags or coherence, a PDS is usually the first product to make. In this example the selected output type is `POWER`, and the output filename is `event.pds`.

The routine can also rebin the data in time. In the example from the original manual, the event data have a time resolution of 16 microseconds. Rebinning by a factor of 4 reduces the highest frequency of interest while retaining enough frequency range for high-frequency features.

The next choice is the number of time bins per transform, or equivalently the duration of each uninterrupted data stretch. If a gap is reached before the requested length is accumulated, the partial stretch is discarded and a new one starts after the gap. This prevents gaps from entering individual Fourier transforms. The routine accepts either the number of bins or the duration in seconds; using a real number such as `32.0` requests a 32-second transform.

```
---------------------------------------------------------------
Program gh_xte     Tue Jan 3 15:26:00 2012     User: belloni
---------------------------------------------------------------
Working directory              :
/Users/belloni/data/xte/u1636/realtime/96087-01-84-10/pca
Input filename                 :
/Users/belloni/data/xte/u1636/realtime/96087-01-84-10/pca/@event.lis
Instrument                     : XTE/PCA
Observation ID                 : 96087-01-84-10
Source name                    : 4U_1636-53
Start date of observation      : 2011-12-29T04:01:36

Channels for accumulation      :    0-249
Type of output                 :    POWER
Output filename                :    event.pds
Time resolution (s)            :    1/8192
Nyquist frequency (Hz)         :    4096.0000
Number of points per FFT       :    262144
Corresponding to length (s)    :    32.000000
Total number of FFTs           :    64
Time step                      :    32.000000
---------------------------------------------------------------
```

3.3 Inspecting the Product

The file `event.pds` contains a sequence of PDS, one per selected uninterrupted data stretch. It also stores the corresponding count rate for each transform and auxiliary information used by later GHATS routines.

The quickest overview is:

```
Ghats> ghats_all,'event.pds'
GH_info -----------------------------------------------

PDS file name        :        event.pds

Observatory          :        XTE
```

```
Instrument           :        PCA
Target               :        4U_1636-53

N. of trafos    :   64
Trafo duration :    32.000000
N. of powers    :   262144
Start-end chans.:   0-249
Starting mjd    :   55924.168
Time step       :   0.031250000
Barycentered    :   NO
Spectral bins   :   3
Background      :   NO
-------------------------------------------------------------
```

`ghats_all` prints the basic header information and creates a summary plot with the light curve and the averaged PDS. The same text information can be obtained with:

```
Ghats> gh_info,'event.pds'
```

> **Note.** Spaces between parts of IDL/GDL commands are harmless but not necessary. The examples in this manual sometimes include spaces for readability.

3.4 Extracting and Plotting the Light Curve

The light curve stored in the PDS file can be extracted and plotted with:

```
Ghats> gh_licu,'event.pds',time,licu
Ghats> gh_plot_licu,time,licu
```

Here `time` and `licu` are arbitrary IDL/GDL variable names chosen by the user.

3.5 Averaging, Rebinning, and Plotting the PDS

The workhorse routine for reading and averaging PDS files is `ghx`. With the default selections, the average PDS can be extracted as:

```
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson
  64 power spectra selected
```

This reads the average PDS into three arrays: `nu` for frequency, `pow` for power, and `pow_e` for the power error. The `/POISSON` keyword requests computation and subtraction of the Poissonian component.

The PDS can then be rebinned:

```
Ghats> gh_reb,nu,pow,pow_e,-100,x,y,ye
```

This writes the rebinned frequency, power, and error arrays to `x`, `y`, and `ye`. Finally:

```
Ghats> gh_plot_power,x,y,ye
```

3.6 Custom PDS Selections

The light curve can be used to select only transforms with count rates above a chosen value. In the legacy example, the mean count rate is:

```
Ghats> print,mean(licu)
  344.632
```

To average only PDS with count rates above this value:

```
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson,rate=[344.632,10000]
  33 power spectra selected
```

Selections can also be combined with a time interval:

```
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson,rate=[344.632,10000],time=[0,500]
  10 power spectra selected
```

or with an index interval:

```
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson,index=[13,34]
  22 power spectra selected
```

> **Note.** GHATS command selections use indices starting from 1, not from 0.

One can also pass an explicit selection array, for instance to select only odd indices:

```
Ghats> odd_indices = 2*indgen(32)+1
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson,rate=[344.632,10000], $
       time=[0,500],sel=odd_indices
  7 power spectra selected
```

> **Note.** The legacy Word/PDF extraction of this line appears malformed. The version above gives the 1-based odd indices from 1 to 63.

Finally, `ghx` can return an rms-normalized PDS if the source and background rates are supplied through the appropriate keyword. The background rate must be estimated by the user.

```
Ghats> ghx,'event.pds',nu,pow,pow_e,/poisson,rate=[344.632,10000], $
       time=[0,500],sel=odd_indices,rms=17.36
  7 power spectra selected
```

> **Note.** The scientific definitions and normalizations in this section are inherited from the GHATS routines. This chapter is only a LaTeX conversion prototype; it does not redefine the PDS, FFT, or normalization conventions.

Chapter 4

Producing PDS and FFT Products

The first GHATS step for most analyses is to turn mission event data, binned data, or an external light curve into a native GHATS timing product. There are two closely related products:

- a `.pds` file, containing one power spectrum for each accepted uninterrupted data stretch;
- a `.fft` file, containing the corresponding complex Fourier amplitudes.

The two files are made by the same family of mission-specific production routines. They use the same transform lengths, time binning, channel selection, GTI handling, and GHATS header conventions. The difference is only what is stored after each transform has been computed: powers for a `.pds` file, or complex Fourier amplitudes for a `.fft` file.

> **Note.** Use a `.pds` file when the next step is ordinary power-spectral analysis. Use a `.fft` file when the next step needs phase information, for example cross spectra, lags, coherence, or bispectral products.

4.1 Common Calling Sequence

Most mission routines have the same basic interface:

```
Ghats> gh_xte
Ghats> gh_xte, '#parameters.par'
Ghats> gh_xte, infile, outtype, channels, treb, npds, outfile
```

The first form runs interactively. The second form reads a parameter file. The third form gives the main parameters directly on the IDL/GDL command line. For a different mission, replace `gh_xte` by the appropriate routine, for instance `gh_nicer`, `gh_ep`, or `gh_nustar`.

`infile` Input science file, `@` metafile, or `@@` meta-metafile. A metafile contains one input filename per line.

`outtype` Output type. Use `'POWER'` for a `.pds` file or `'FFT'` for a `.fft` file.

`channels` Two-element channel interval, for example `[0,35]`. The meaning of the channel is mission dependent: RXTE/PCA uses PCA channels, while event-list missions normally use PI or PHA channels.

`treb` Integer rebinning factor in time before computing the transforms. A value of 1 keeps the native time resolution.

npds Transform size. If this is an integer or long integer, it is interpreted as the number of rebinned light-curve bins per transform. If this is a real number, it is interpreted as a duration in seconds.

outfile Output filename, normally ending in `.pds` or `.fft`.

For example:

```
Ghats> gh_xte, '@event.lis', 'POWER', [0,249], 1, 8192, 'fullband.pds'
Ghats> gh_xte, '@event.lis', 'FFT',   [0,13],  1, 8192, 'soft.fft'
Ghats> gh_xte, '@event.lis', 'FFT',   [14,249],1, 8192, 'hard.fft'
```

The two `.fft` files in the last two examples can then be used for cross-spectral work, provided they contain the same accepted transforms in the same order.

4.2 Interactive and Parameter-File Modes

When called without arguments, a production routine asks the user for the same choices that appear in the command-line form: input file, output type, channel range, time rebinning, transform length, and output filename. This is often the safest way to make a first product from an unfamiliar observation, because the routine prints the channel information, time resolution, and final transform summary as it goes.

A parameter file gives the same six arguments to the routine. The command is called with a leading `#`:

```
Ghats> gh_xte, '#mysource.par'
```

This is useful when the same production command must be repeated exactly, or when several energy bands are being produced from the same input data. The parameter-file route should preserve the same choices as the explicit command-line route; if two products will later be compared transform by transform, make the choices identical except for the channel range.

4.3 Input Lists

The input filename may point to a single file or to a list. GHATS uses a leading `@` to identify a metafile:

```
$ ls *evt* > event.lis
Ghats> gh_nicer, '@event.lis', 'POWER', [30,1200], 1, 4096, 'nicer.pds'
```

A leading `@@` identifies a meta-metafile, that is, a file containing names of metafiles. This convention is inherited from the RXTE routines and is useful when one observation contains several groups of homogeneous input files.

Files grouped in one list should be compatible: same data mode or event-file layout, same time resolution, same relevant detector setup, and the same energy-channel convention. GTIs and gaps may differ from file to file; the production routine constructs transforms only from accepted continuous stretches.

4.4 Choosing the Transform Length

The transform length determines the frequency grid. If N_{pds} rebinned bins enter each transform and the rebinned time resolution is Δt, then the transform duration is

$$T_{\mathrm{seg}} = N_{\mathrm{pds}}\,\Delta t.$$

The frequency resolution is $1/T_{\mathrm{seg}}$, and the Nyquist frequency is $1/(2\Delta t)$. In GHATS files, the DC bin is not stored and the positive Fourier bins through the Nyquist bin are stored.

The production routines also accept the transform length in seconds. For example, if the data after rebinning have a time resolution of 1/8192 s, then these two calls request the same transform duration:

```
Ghats> gh_xte, '@event.lis', 'POWER', [0,249], 1, 262144L, 'event.pds'
Ghats> gh_xte, '@event.lis', 'POWER', [0,249], 1, 32.0,    'event.pds'
```

Note. The number of points per transform should be a power of two. Long transform lengths give finer frequency resolution but require longer uninterrupted data stretches. Shorter transform lengths keep more stretches when the observation contains many gaps.

4.5 Channels and Standard Bands

The main `channels` argument selects the channel range accumulated into the timing product itself. Most production routines also accept `/BANDS`. This keyword defines three auxiliary channel bands stored in the GHATS product and used later by routines such as `gh_colors`.

```
Ghats> gh_xte, '@event.lis', 'POWER', [0,249], 1, 4096, 'event.pds', $
       bands=[3,6,7,14,15,20]
```

The six numbers define three intervals: `3--6`, `7--14`, and `15--20`. They are instrument channels, not automatically calibrated physical energy bands. For missions such as Einstein Probe, the default split is a technical PI-channel split unless the user supplies a more appropriate one.

4.6 GTIs, Barycentered Times, and Sliding Windows

Several common keywords control how the input data are segmented:

`/GTI` Intersect the data GTIs with user-supplied GTIs. The user GTI file may be an ASCII file with start and stop times or a standard GTI FITS file, depending on the mission routine.

`/BARY` Read barycentered times from the appropriate barycentered time column, normally `BARYTIME`. The user is responsible for ensuring that the column exists and is appropriate.

`/SLIDING` Use overlapping transforms. If `SLIDING=4`, consecutive transforms are shifted by one quarter of the transform duration. This can be useful diagnostically, but the averaged PDS obtained from overlapping stretches does not have the same simple statistical interpretation as an average of independent transforms.

4.7 Window Functions

The production routines accept `/WIND` and `WPAR`, which are passed to the GHATS windowing machinery. If no window is requested, the default is a boxcar window. A non-boxcar window changes the effective weighting of the light curve before the Fourier transform and should therefore be recorded when reporting a product.

4.8 Mission-Specific Routines

Routine	Mission	Purpose
`gh_xte`	RXTE/PCA	Produce GHATS PDS or FFT files from RXTE/PCA data.
`gh_nicer`	NICER	Produce GHATS PDS or FFT files from NICER event data.
`gh_ep`	Einstein Probe	Produce GHATS PDS or FFT files from Einstein Probe event data.
`gh_nustar`	NuSTAR	Produce GHATS PDS or FFT files from NuSTAR event data.
`gh_xmm`	XMM	Produce GHATS PDS or FFT files from XMM event data.
`gh_swift`	Swift/XRT	Produce GHATS PDS or FFT files from Swift/XRT event data.
`gh_ixpe`	IXPE	Produce GHATS PDS or FFT files from IXPE event data.
`gh_laxpc`	AstroSat/LAXPC	Produce GHATS PDS or FFT files from AstroSat/LAXPC data.
`gh_laxpc_misra`	AstroSat/LAXPC	Produce GHATS PDS or FFT files from AstroSat/LAXPC event data.
`gh_czti`	AstroSat/CZTI	Produce GHATS PDS or FFT files from AstroSat/CZTI event data.
`gh_huiyan`	Huiyan/HXMT	Produce GHATS PDS or FFT files from Huiyan/HXMT data.
`gh_huiyan_le`	Huiyan/HXMT LE	Produce GHATS PDS or FFT files from Huiyan/HXMT LE data.
`gh_huiyan_me`	Huiyan/HXMT ME	Produce GHATS PDS or FFT files from Huiyan/HXMT ME data.
`gh_huiyan_he`	Huiyan/HXMT HE	Produce GHATS PDS or FFT files from Huiyan/HXMT HE data.
`gh_ascii_lcfft`	Contributed	Produce GHATS PDS or FFT files from an ASCII light curve.

Each routine can be queried from GHATS:

```
Ghats> gh_xte, /help
Ghats> gh_nicer, /help
Ghats> gh_ep, /help
```

> **Note.** Some mission front ends use the IDL common block name `COMMON DATI` with different internal layouts. The safe practice is to use a fresh GHATS/IDL session when switching between mission families, especially between the RXTE/event-list, Huiyan/HXMT, LAXPC, and CZTI production paths. In IDL, even asking for `/HELP` can compile the routine and therefore check the mission-specific `COMMON DATI` layout.

4.9 RXTE/PCA

`gh_xte` is the original GHATS production routine and supports RXTE binned, single-bit, event, and GoodXenon-style inputs. It also supports `/TURBO`, which uses the faster RADPS path where available, and `/BANDS` for the three auxiliary Standard-2-like rate bands.

```
Ghats> gh_xte, '@event.lis', 'POWER', [0,249], 1, 4096, 'rxte_full.pds'
Ghats> gh_xte, '@event.lis', 'FFT',   [0,13],  1, 4096, 'rxte_soft.fft'
```

For RXTE/PCA PDS products, GHATS can store and later use PCA-specific information relevant to Poisson-noise estimates. The standard PDS values written by the usual production path are in Leahy normalization. The `.fft` products store raw complex Fourier amplitudes; normalizations are applied by later analysis routines.

4.10 Event-List Missions

Several newer routines follow the same event-list template: `gh_nicer`, `gh_ep`, `gh_nustar`, `gh_xmm`, `gh_swift`, and `gh_ixpe`. Their common command-line form is:

```
Ghats> gh_nicer, infile, outtype, channels, treb, npds, outfile
```

The most important mission-dependent details are the event time column, the channel column, the native time system, and the default auxiliary bands. The routine help should be checked before first use:

```
Ghats> gh_nicer, /help
Ghats> gh_ep, /help
```

Examples:

```
Ghats> gh_nicer, 'ni_clean.evt', 'FFT', [30,1200], 10000, 32768, 'ni.fft'
Ghats> gh_ep, 'fxt_clean.fits', 'POWER', [0,1023], 1, 4096, 'fxt.pds'
```

> **Note.** `gh_nicer` has the keyword `/NONOISYDET` for NICER-specific noisy-detector filtering. This is not a generic event-list option and is not used by `gh_ep`.

4.11 AstroSat and Huiyan/HXMT

AstroSat and Huiyan/HXMT have mission-specific front ends because their data layouts and detector selections differ from the simpler event-list template. The output products, however, are ordinary GHATS `.pds` and `.fft` files and can be read by the same downstream routines.

For AstroSat:

```
Ghats> gh_laxpc, infile, outtype, channels, treb, npds, outfile
Ghats> gh_laxpc_misra, infile, outtype, channels, units, layers, treb, npds,
    outfile
Ghats> gh_czti, infile, outtype, channels, treb, npds, outfile
```

For Huiyan/HXMT:

```
Ghats> gh_huiyan, infile, outtype, channels, grades, treb, npds, outfile
Ghats> gh_huiyan_le, infile, outtype, channels, treb, npds, outfile
Ghats> gh_huiyan_me, infile, outtype, channels, treb, npds, outfile
Ghats> gh_huiyan_he, infile, outtype, channels, treb, npds, outfile
```

Use the routine-specific `/HELP` output for the exact channel, detector, unit, and layer arguments.

4.12 ASCII Light Curves

`gh_ascii_lcfft` is a contributed routine for producing a GHATS `.pds` or `.fft` file from an ASCII light curve with columns

```
time(s)   counts_or_rate   error
```

The error column is accepted for compatibility with common light-curve formats, but is not used in the Fourier calculation. By default, the second column is interpreted as counts per original light-curve bin. If it is a count rate, use `/RATE`.

```
Ghats> gh_ascii_lcfft, 'lc.txt', 'POWER', 1, 4096, 'lc.pds', dt=0.016d0
Ghats> gh_ascii_lcfft, 'lc_rate.txt', 'FFT', 1, 8192, 'lc.fft', $
       dt=0.001d0, /rate
```

The routine detects gaps in the input time column and does not Fourier transform across them. Rows with non-finite times or rates are treated as boundaries rather than interpolated over.

4.13 After Production

After making a product, inspect it before using it in later analysis:

```
Ghats> gh_info, 'event.pds'
Ghats> ghats_all, 'event.pds'
```

For `.fft` files, `gh_info` is the appropriate header check. `ghats_all` can also read an FFT product and compute the PDS panel from the individual FFT records, but the light-curve panel is not reliable for FFT input.

The main downstream readers are:

- `ghx`, to read and average `.pds` products;
- `ghx_fft`, to read and average `.fft` products;
- `gh_licu`, to extract the stored light curve;
- `gh_cs` and related cross-spectral routines, to work from matching FFT products.

For the exact binary layout, frequency convention, FFT sign convention, and normalization details, see Appendix B.

Chapter 5

Reading, Averaging, and Rebinning

Once a GHATS timing product has been written, the next step is usually to read selected transforms, average them, subtract or estimate the noise level when appropriate, and optionally rebin the result. This chapter describes the routines used for that stage:

- `ghx`, for `.pds` files;
- `ghx_fft`, for `.fft` files;
- `gh_reb`, for linear or logarithmic rebinning;
- auxiliary readers such as `gh_licu`, `gh_colors`, and `gh_hk_xte`.

The routines in this chapter do not remake Fourier transforms. They read products already written by the production routines described in the previous chapter.

5.1 `ghx`: Averaging PDS Files

`ghx` is the main routine for reading a GHATS `.pds` file and returning an averaged power spectrum:

```
Ghats> ghx, 'file.pds', frequency, power, power_err
```

The output arrays are:

`frequency` Frequency array, in Hz.

`power` Averaged power spectrum.

`power_err` Error estimate for the averaged power spectrum.

By default, all transforms in the file are selected and no noise level is subtracted:

```
Ghats> ghx, 'event.pds', nu, pow, powe
  64 power spectra selected
WARNING: raw PDS returned; no noise level subtracted.
Use /POISSON, POISSON=[f1,f2], or MANUAL_POI=p to subtract one.
```

This warning is intentional. It reminds the user that the returned PDS is the raw average from the file unless a noise prescription is requested.

5.2 Selecting Transforms

The same selection keywords are used by `ghx` and `ghx_fft`:

`INDEX=[i1,i2]` Select transforms with internal indices between `i1` and `i2`.

`TIME=[t1,t2]` Select transforms whose start and end times fall inside the interval `t1--t2`, in seconds from the file start.

`RATE=[r1,r2]` Select transforms whose stored count rate lies in the interval `r1--r2`.

`SEL=sel` Select an explicit array of transform indices.

Selections can be combined:

```
Ghats> ghx, 'event.pds', nu, pow, powe, time=[0,500], $
       rate=[300,10000], index=[0,31]
```

> **Note.** In the current `ghx` and `ghx_fft` code, `INDEX` and `SEL` are compared with the internal transform counter, which starts from 0. Thus `INDEX=[0,9]` selects the first ten transforms.

If no transforms satisfy the selection, `ghx` prints a warning and the returned arrays should not be used.

5.3 Reading Several Compatible PDS Files

`ghx` also accepts a metafile as input. The filename must begin with `@`, and the metafile must contain one `.pds` filename per line:

```
$ ls obs*_soft.pds > pds_files.lis
Ghats> ghx, '@pds_files.lis', nu, pow, powe, poisson=[400,800]
```

The files are treated as if their transforms came from a single longer PDS file, but only after compatibility checks. The current checks require the same transform size, frequency resolution, channel range, proliferation flag, barycenter flag, spectral-bin layout, background flag, GHATS mode flags, window flag, and VLE flag. If the files are not listed in time order, `ghx` prints a warning and processes them sorted by the header start time. If their times overlap or repeat, the routine stops.

> **Note.** Metafile input for `ghx` is for PDS files only. The files listed inside the metafile must already be GHATS `.pds` products.

5.4 Noise Subtraction in `ghx`

`ghx` supports three mutually exclusive noise-subtraction modes.

5.4.1 `/POISSON`

With `/POISSON`, `ghx` computes and subtracts the legacy Zhang-style Poisson level:

```
Ghats> ghx, 'event.pds', nu, pow, powe, /poisson
```

This is the historical RXTE/PCA path. It uses the count rate, VLE information, number of active detectors, and the transform length stored in the PDS product.

5.4.2 POISSON=[f1,f2]

With a two-element POISSON keyword, ghx estimates a constant noise level from the requested frequency range and subtracts it from the whole PDS:

```
Ghats> ghx, 'event.pds', nu, pow, powe, poisson=[400,800]
```

The range is clipped to the available non-Nyquist frequencies. The Nyquist bin is excluded from this estimate because its statistics differ from the other positive-frequency bins for a real time series. If the requested range has no valid overlap with the PDS frequency grid, the routine stops.

5.4.3 MANUAL_POI=p

With MANUAL_POI, the user supplies a constant level in the input-file PDS units:

```
Ghats> ghx, 'event.pds', nu, pow, powe, manual_poi=2.0
```

This is useful when the noise level has been measured or fitted elsewhere. The value is interpreted before any optional rms^2 conversion.

> **Note.** Use only one noise option at a time: /POISSON, POISSON=[f1,f2], or MANUAL_POI=p. The routine stops if more than one is requested.

5.5 RMS-Squared Conversion

If the RMS keyword is given, ghx converts the PDS to rms^2 units after the selected transforms have been averaged. The value passed through RMS is the background rate:

```
Ghats> ghx, 'event.pds', nu, pow, powe, poisson=[400,800], rms=0.001
```

The routine uses the average total rate of the selected transforms and stops if the supplied background rate is larger than the total source-plus-background rate. If a manually supplied constant noise level is used together with RMS, that constant must be in the original input-file PDS units, not in the rms-normalized units.

5.6 ghx_poisrange

ghx_poisrange is a contributed routine kept for workflows that estimate an empirical residual constant from a frequency range after optional rms^2 conversion:

```
Ghats> ghx_poisrange, 'event.pds', nu, pow, powe, rms=0.001, $
       poisrange=[400,800]
```

The newer ghx interface can also estimate and subtract a constant with POISSON=[f1,f2]. The distinction is the order of operations: ghx interprets the constant in the input PDS units before optional rms^2 conversion, while ghx_poisrange was written to reproduce the older interactive practice of estimating the residual level from an already rms-normalized PDS.

5.7 ghx_fft: Averaging FFT Files

ghx_fft reads a GHATS .fft file and averages the complex Fourier amplitudes:

```
Ghats> ghx_fft, 'file.fft', frequency, fft, fft_err
```

The output `fft` array is complex. It is not a power spectrum, and it is not normalized as Leahy or rms power. The stored `.fft` payload is the raw complex Fourier amplitude written by the production routine; later cross-spectral routines apply the normalizations they need.

The same selection keywords are available:

```
Ghats> ghx_fft, 'soft.fft', nu, soft_fft, soft_fft_err, index=[0,99]
Ghats> ghx_fft, 'hard.fft', nu, hard_fft, hard_fft_err, index=[0,99]
```

For cross spectra, lags, coherence, or bispectra, separate FFT files should have the same time resolution, transform length, frequency grid, and selected transform list. If two energy bands were produced from the same input data with the same parameters, changing only the channel range, their transforms should normally correspond one-to-one.

5.8 `gh_reb`: Rebinning Arrays

`gh_reb` rebins arrays with errors:

```
Ghats> gh_reb, frequency, power, power_err, irf, reb_frequency, $
       reb_power, reb_power_err
```

The sign of `irf` selects the rebinning mode:

`irf > 0` Linear rebinning. Adjacent bins are combined in groups set by the rebin factor.

`irf < 0` Logarithmic rebinning. For example, `irf=-20` gives bins whose width grows by a factor exp(1/20) from one bin to the next.

The most common PDS workflow is:

```
Ghats> ghx, 'event.pds', nu, pow, powe, /poisson
Ghats> gh_reb, nu, pow, powe, -100, x, y, ye
Ghats> gh_plot_power, x, y, ye
```

`gh_reb` is a general array rebinning wrapper around the GHATS rebinning routine. It does not read headers and does not know whether the input came from a PDS, a cross spectrum, or another array with errors.

5.9 Light Curves

`gh_licu` extracts the count-rate light curve stored in a GHATS PDS file:

```
Ghats> gh_licu, 'event.pds', times, rate
Ghats> gh_licu, 'event.pds', times, rate, deltat=dt, mjd=mjd0
```

The output `times` array is in seconds from the file start. The `rate` array contains one rate value per transform. The keyword output `DELTAT` returns the time spacing between adjacent light-curve points, and `MJD` returns the starting MJD from the file header.

> **Note.** `gh_licu` reads the light curve stored in a PDS product. It is not a general event-file light-curve extractor.

5.10 Counting Transforms

`gh_nspectra` reads the number of transforms stored in the PDS header:

```
Ghats> gh_nspectra, 'event.pds', ntrafos
Ghats> print, ntrafos
```

This is a quick way to check how many spectra are available before deciding on `INDEX`, `TIME`, or `RATE` selections.

5.11 Colors and Housekeeping

`gh_colors` reads the three auxiliary rate curves stored in a PDS file and returns two hardness ratios:

```
Ghats> gh_colors, 'event.pds', times, hr1, hr2, rate1, rate2, rate3
```

The rates correspond to the three channel bands requested during PDS/FFT production with `/BANDS`. The hardness ratios are:

$$hr1 = \frac{rate2}{rate1}, \qquad hr2 = \frac{rate3}{rate1}.$$

For RXTE/PCA products, `gh_hk_xte` reads housekeeping-like series stored with each transform:

```
Ghats> gh_hk_xte, 'event.pds', times, vle, ndet, poiss
```

The outputs are time, VLE rate per detector, number of active detectors, and the Poisson level stored for each transform. The helper routine `gh_getvle` reads the RXTE/PCA VLE time parameter directly from an FH5a housekeeping file:

```
Ghats> gh_getvle, 'FH5a_file', i_vle, /nodialog
```

5.12 Dynamic PDS Extraction

`gh_dyn` reads a PDS file into a time-frequency image:

```
Ghats> gh_dyn, 'event.pds', time, rate, frequency, dynimage
Ghats> gh_dyn, 'event.pds', time, rate, frequency, dynimage, $
       index=[0,99], frebin=-100, trebin=4
```

The output `dynimage` is a dynamic PDS, with one axis corresponding to time and the other to frequency after optional rebinning. The full discussion of dynamic spectra and FITS export is in Chapter 8.

5.13 A Typical PDS Preparation Sequence

A compact sequence for making an analysis-ready average PDS is:

```
Ghats> gh_info, 'event.pds'
Ghats> gh_licu, 'event.pds', time, rate
Ghats> ghx, 'event.pds', nu, pow, powe, poisson=[400,800], $
       rate=[300,10000]
Ghats> gh_reb, nu, pow, powe, -100, x, y, ye
Ghats> gh_plot_power, x, y, ye
```

For an FFT-based analysis, the analogous first check is:

```
Ghats> gh_info, 'soft.fft'
Ghats> gh_info, 'hard.fft'
Ghats> ghx_fft, 'soft.fft', nu, soft_fft, soft_fft_err
Ghats> ghx_fft, 'hard.fft', nu, hard_fft, hard_fft_err
```

In practice, `ghx_fft` is mostly a diagnostic reader. Cross-spectral and bispectral routines normally read the FFT products themselves so that they can apply the GHATS conventions consistently.

Chapter 6

Plotting

GHATS includes small plotting routines for the most common quick-look timing products. They are intentionally simple: the routines plot arrays that have already been read or computed by other GHATS commands, and they use the current IDL/GDL graphics device unless `/PS` is requested.

The typical pattern is:

```
Ghats> ghx, 'event.pds', nu, pow, powe, /poisson
Ghats> gh_reb, nu, pow, powe, -100, x, y, ye
Ghats> gh_plot_power, x, y, ye
```

For a first inspection of a PDS file, `ghats_all` is often even faster because it reads the file, prints the header information, extracts the stored light curve, averages the PDS, and makes a summary plot.

6.1 Common Plot Arguments

Most plotting routines use the same optional axis limits:

```
Ghats> gh_plot_power, frequency, power, power_err
Ghats> gh_plot_power, frequency, power, power_err, x1, x2
Ghats> gh_plot_power, frequency, power, power_err, x1, x2, y1, y2
```

The first form lets the routine choose both axes. The second form fixes the horizontal range. The third form fixes both horizontal and vertical ranges. The meaning of `x1,x2` and `y1,y2` depends on the plotted quantity, but the convention is always minimum then maximum.

Most routines also accept `/PS`. This writes a PostScript file in the current directory and then restores the previous graphics device. The output filename is fixed by the routine, for example `gh_power.ps` for `gh_plot_power`.

6.2 PDS Plots

`gh_plot_power` plots $P(f)$ in log-log form:

```
Ghats> gh_plot_power, frequency, power, power_err
Ghats> gh_plot_power, frequency, power, power_err, 0.1, 64.0
Ghats> gh_plot_power, frequency, power, power_err, 0.1, 64.0, 1.0e-4, 10.0
```

The arguments are:

`frequency` Frequency array, in Hz.

`power` Power array.

`power_err` Error array for the power.

`x1,x2` Optional frequency range to plot.

`y1,y2` Optional power range to plot.

Only positive power values are plotted, because the display is logarithmic in the vertical direction. This matters after Poisson subtraction: bins with zero or negative powers are silently omitted from the log-log plot.

`gh_plot_nupower` plots $fP(f)$ instead:

```
Ghats> gh_plot_nupower, frequency, power, power_err
Ghats> gh_plot_nupower, frequency, power, power_err, 0.1, 64.0
```

The input power and error arrays are multiplied by frequency before plotting. The optional `y1,y2` range is therefore a range in $fP(f)$, not in $P(f)$.

> **Note.** `gh_plot_power` and `gh_plot_nupower` do not change the input arrays. The multiplication by frequency in `gh_plot_nupower` is only used for the plotted values.

6.3 Overplotting PDS

The overplot routines add another spectrum to the current plot:

```
Ghats> gh_plot_power, nu1, pow1, err1
Ghats> gh_oplot_power, nu2, pow2, err2
```

For $fP(f)$:

```
Ghats> gh_plot_nupower, nu1, pow1, err1
Ghats> gh_oplot_nupower, nu2, pow2, err2
```

The first command establishes the axes; the overplot command uses the existing graphics window. Therefore the arrays added with `gh_oplot_power` or `gh_oplot_nupower` should lie inside the current axis ranges, or they will not be visible.

6.4 Light-Curve Plots

`gh_plot_licu` plots a light curve as rate versus time:

```
Ghats> gh_licu, 'event.pds', time, rate
Ghats> gh_plot_licu, time, rate
Ghats> gh_plot_licu, time, rate, 0.0, 1000.0
Ghats> gh_plot_licu, time, rate, 0.0, 1000.0, 200.0, 500.0
```

Here `x1,x2` are times in seconds and `y1,y2` are count-rate limits. The routine draws a stepped light curve without error bars.

`gh_plot_licu_nogaps` uses the same rate array but plots it against PDS index rather than time:

```
Ghats> gh_plot_licu_nogaps, time, rate
```

The `time` array is used only to identify gaps. The horizontal axis is the PDS index, starting from 0, and vertical dashed lines mark gaps inferred from the spacing of adjacent times. This is useful when the real elapsed time contains long gaps that would compress the good intervals into narrow regions of the plot.

In short, the ordinary light-curve plot shows elapsed seconds, while the no-gaps plot shows the sequence of PDS transforms and marks where gaps occurred.

6.5 Housekeeping Plots

`gh_plot_hk_xte` is a convenience plot for RXTE/PCA housekeeping series stored in a GHATS PDS file:

```
Ghats> gh_plot_hk_xte, 'event.pds'
Ghats> gh_plot_hk_xte, 'event.pds', /ps
```

Internally it calls `gh_hk_xte`, then plots the VLE rate, number of active detectors, and stored Poisson level as a multi-panel diagnostic. With `/PS`, the output file is `gh_hk_xte.ps`.

6.6 Cross-Spectral Plots

Cross-spectral analysis routines return arrays that can be plotted directly. For lag spectra, use `gh_plot_lag`:

```
Ghats> gh_plot_lag, frequency, lag, lag_err
Ghats> gh_plot_lag, frequency, lag, lag_err, 0.1, 64.0
Ghats> gh_plot_lag, frequency, lag, lag_err, 0.1, 64.0, -1.0, 1.0, /lin
```

By default, the routine labels the vertical axis as phase lag in radians. If `/TIME` is set, the vertical axis is labelled as time lag in seconds:

```
Ghats> gh_plot_lag, frequency, time_lag, time_lag_err, /time, /lin
```

The horizontal axis is logarithmic. The keyword `/LIN` keeps the vertical axis linear. Without `/LIN`, the routine requests a logarithmic vertical axis, which is only appropriate when the plotted lag values are positive. A horizontal zero-lag line is drawn for reference. The `/HYP` keyword plots the hyperbolic sine of the lag values, a legacy option intended to give a log-like display when negative lags are present.

For coherence spectra, use `gh_plot_coh`:

```
Ghats> gh_plot_coh, frequency, coherence, coherence_err
Ghats> gh_plot_coh, frequency, coherence, coherence_err, 0.1, 64.0, 0.0, 1.1
```

The default vertical range is 0 to 1.1. The horizontal axis is logarithmic, and error bars are drawn using the IDL or GDL plotting helper available in the current session. Both `gh_plot_lag` and `gh_plot_coh` also accept `/HELP`.

6.7 PostScript Output

For plotting routines that support it, `/PS` writes a file with a fixed name:

Routine	PostScript file
`gh_plot_power`	`gh_power.ps`
`gh_plot_nupower`	`gh_nupower.ps`
`gh_plot_licu`	`gh_licu.ps`

Routine	PostScript file
gh_plot_licu_nogaps	gh_licu.ps
gh_plot_hk_xte	gh_hk_xte.ps
gh_plot_lag	gh_lag.ps
gh_plot_coh	gh_coh.ps

Because these filenames are fixed, running the same plotting command again in the same directory overwrites the previous PostScript file. Rename important plots after making them if several versions need to be kept.

6.8 A Compact Plotting Workflow

A common exploratory sequence is:

```
Ghats> gh_info, 'event.pds'
Ghats> gh_licu, 'event.pds', time, rate
Ghats> gh_plot_licu, time, rate
Ghats> ghx, 'event.pds', nu, pow, powe, poisson=[400,800]
Ghats> gh_reb, nu, pow, powe, -100, x, y, ye
Ghats> gh_plot_power, x, y, ye
Ghats> gh_plot_nupower, x, y, ye
```

For publication-quality figures, these quick-look plots are often the starting point rather than the final product. They are most useful for checking that the selected transforms, frequency range, noise subtraction, and rebinning are reasonable before carrying the arrays into more specialised plotting scripts.

Chapter 7

Cross Spectra, Lags, and Coherence

Cross-spectral analysis in GHATS starts from two compatible `.fft` files. The two files should normally be produced from the same event data, with the same time bin, transform length, GTI selection, and start time, changing only the channel or energy selection. The routines then average the complex cross spectrum and return phase or time lags, coherence, and, in the newer interface, the real and imaginary parts of the cross spectrum.

The routines described here are:

- `gh_cross`, the older lag/coherence reader;
- `gh_cross_re_im_new`, the lower-level routine that also returns the real and imaginary parts;
- `gh_cs`, the recommended wrapper for most current work;
- `gh_plot_lag` and `gh_plot_coh`, described in the previous chapter, for quick-look plots.

7.1 Lag Sign and Basic Convention

The first filename is the reference band and the second filename is the subject band:

```
Ghats> file_ref = 'soft.fft'
Ghats> file_sub = 'hard.fft'
```

Positive lags mean that the second time series lags the first. By default the lag array is a phase lag in radians. If `/TLAG` is set, the routine divides by $2\pi f$ and returns time lags in seconds:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf

Ghats> gh_cs, file_ref, file_sub, freq, tlag, tlag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, /tlag
```

The rebin factor `irf` follows the usual GHATS convention. A positive integer rebins linearly by that number of bins. A negative value requests logarithmic rebinning; `-100` is a common exploratory choice.

7.2 The Recommended Interface: `gh_cs`

`gh_cs` is the current high-level cross-spectral command. It wraps `gh_cross_re_im_new` and, when `POISSON=[f1,f2]` is given, uses a preliminary pass to estimate the mean real cross-spectrum contribution in the requested high-frequency range. It then subtracts that constant real component in the final pass.

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400,800], rms1=0.001, rms2=0.001, /lowcoh
```

The output arrays are:

freq Rebinned Fourier frequencies, in Hz.

lag Phase lags in radians, or time lags in seconds with **/TLAG**.

lag_err Lag errors.

coh Coherence spectrum.

coh_err Coherence errors.

real, imag Real and imaginary parts of the averaged cross spectrum.

real_err, imag_err Errors on the real and imaginary parts.

modcs Cross-spectrum modulus, $\sqrt{\mathrm{Re}(C)^2 + \mathrm{Im}(C)^2}$.

modcs_err Error on the modulus, propagated using the empirical covariance between the real and imaginary parts.

mw Number of averaged cross vectors contributing to each rebinned bin. It is the number of selected transforms times the number of native frequencies in the rebinned bin.

qf Quality flag for the **/LOWCOH** calculation. Values mark the standard high-power/high-coherence case, the low-coherence case, and bins where the implemented conditions do not allow a useful intrinsic-coherence estimate.

> **Note.** Use **gh_cs** unless you specifically need to control the two passes yourself. It is less error-prone than manually measuring and subtracting the constant real component.

7.3 Compatible FFT Files and Selections

The routines check that the two FFT files are compatible. They require the same time resolution, Fourier length, start time, and number of transforms. The observatory and target keywords must also agree. The current **gh_cross_re_im_new** code warns, but does not stop, if the instrument strings differ.

The selection keywords are the same style as in the PDS/FFT readers:

INDEX=[i1,i2] Select an internal transform-index range.

TIME=[t1,t2] Select transforms by time range.

RATE=[r1,r2] Select transforms by stored count-rate range.

SEL=sel Select an explicit array of transform indices.

The selected transform list should match between the two FFT files. If the files were produced from the same data with the same production parameters, the transform numbers should normally correspond one-to-one.

7.4 Noise Prescriptions for Coherence

`gh_cs` and `gh_cross_re_im_new` distinguish raw measured coherence from intrinsic coherence. The choice is explicit.

7.4.1 `POISSON=[f1,f2]`

`POISSON=[f1,f2]` estimates the PDS noise levels from a frequency range and uses them in the intrinsic-coherence denominator. In `gh_cs`, the same frequency range is also used to estimate the constant real cross-spectrum contribution that is subtracted from the real part:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400,800]
```

The requested range is clipped internally to available non-Nyquist frequencies when necessary. The Nyquist bin is excluded from the noise estimate and from the real-part correction.

7.4.2 `MANUAL_POI=[p1,p2]`

`MANUAL_POI=[p1,p2]` supplies fixed PDS noise levels for the two input bands:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       manual_poi=[2.01,2.03]
```

The two constants are interpreted in the input-file PDS units, before any `RMS1`/`RMS2` conversion. This option affects the coherence calculation only; it does not apply a real-part correction to the cross spectrum.

7.4.3 `/RAWCOH`

`/RAWCOH` returns measured coherence. The observed PDS are used in the denominator and no Poisson or noise correction is applied:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, /rawcoh
```

This is mutually exclusive with `POISSON`, `MANUAL_POI`, and `/LOWCOH`.

> **Note.** If none of `POISSON`, `MANUAL_POI`, or `/RAWCOH` is given, RXTE/PCA data use the historical Zhang-style Poisson correction. For other instruments the routine returns raw coherence and prints a warning suggesting `POISSON=[f1,f2]` or `MANUAL_POI=[p1,p2]` for intrinsic coherence.

7.5 The `/LOWCOH` Option

`/LOWCOH` changes the coherence-error calculation. It uses the Vaughan–Nowak low-measured-coherence formalism where the implemented conditions indicate that the ordinary high-coherence approximation is not the appropriate one. The returned `qf` array records which path was used for each frequency bin.

Some bins can fall outside the implemented useful range. In that case the routine marks the corresponding internal error term as undefined. In IDL this can print:

```
% Program caused arithmetic error: Floating illegal operand
```

When it appears immediately after a `/LOWCOH` run, this message is normally the result of marking those undefined bins. Other floating-point messages should still be investigated.

7.6 RMS Units for the Cross Spectrum

Without `RMS1` and `RMS2`, the returned real and imaginary parts of the cross spectrum are in the native Leahy-like units used internally by the FFT products. If both background rates are supplied, the routine converts `real`, `imag`, their errors, and the modulus to rms units:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400,800], rms1=0.001, rms2=0.001
```

Both background rates must be supplied together. The first value corresponds to the first FFT file and the second value to the second FFT file.

7.7 Rotating the Cross Vector

The `/ROTATE` keyword rotates the returned real and imaginary parts of the cross spectrum by $\pi/4$:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400,800], /rotate
```

This can be useful for fitting the real and imaginary parts when the unrotated imaginary part is small or changes sign. The rotation does not change the modulus, and it does not change the returned lag or coherence arrays. If the rotated real and imaginary parts are fitted later, the same rotation must be included when comparing the model to those fitted components.

7.8 Using `gh_cross_re_im_new` Directly

`gh_cross_re_im_new` exposes the lower-level calculation:

```
Ghats> gh_cross_re_im_new, file_ref, file_sub, freq, lag, lag_err, $
       coh, coh_err, -100, real, real_err, imag, imag_err, $
       modcs, modcs_err, mw, qf
```

It is useful when the real-part correction has to be controlled manually. For example:

```
Ghats> gh_cross_re_im_new, file_ref, file_sub, freq, lag, lag_err, $
       coh, coh_err, -100, real, real_err, imag, imag_err, $
       modcs, modcs_err, mw, qf
Ghats> w = where(freq ge 400 and freq le 800)
Ghats> poivalue = mean(real[w])
Ghats> gh_cross_re_im_new, file_ref, file_sub, freq, lag, lag_err, $
       coh, coh_err, -100, real, real_err, imag, imag_err, $
       modcs, modcs_err, mw, qf, poivalue=poivalue
```

For ordinary use, the `POISSON=[f1,f2]` option in `gh_cs` performs this two-pass procedure automatically.

7.9 The Older `gh_cross` Routine

`gh_cross` reads two compatible FFT files and returns only frequency, lag, lag error, coherence, and coherence error:

```
Ghats> gh_cross, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, -100
```

It is retained for older workflows. New analyses that need the real and imaginary parts, rms conversion, rotation, the modulus, or the current raw and intrinsic coherence choices should use `gh_cs` or `gh_cross_re_im_new`.

7.10 Writing Cross-Spectral Products for XSPEC

The arrays returned by `gh_cs` can be written as simple OGIP-style PHA/RMF pairs using `gh_xspec`:

```
Ghats> gh_xspec, freq, real, real_err, 'real_soft_hard'
Ghats> gh_xspec, freq, imag, imag_err, 'imag_soft_hard'
Ghats> gh_xspec, freq, lag,  lag_err,  'plag_soft_hard'
Ghats> gh_xspec, freq, coh,  coh_err,  'cohe_soft_hard'
Ghats> gh_xspec, freq, modcs, modcs_err, 'mod_soft_hard'
```

This is a file-format conversion step. It does not refit, rebin, or renormalize the arrays; it writes the values and errors that were passed to it.

7.11 A Compact Workflow

A typical current workflow is:

```
Ghats> gh_nicer, 'event.evt', 'FFT', [30,200], 10000, 32768, 'soft.fft'
Ghats> gh_nicer, 'event.evt', 'FFT', [201,1200], 10000, 32768, 'hard.fft'

Ghats> gh_cs, 'soft.fft', 'hard.fft', freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400,800], rms1=0.001, rms2=0.001, /lowcoh

Ghats> gh_plot_lag, freq, lag, lag_err, /lin
Ghats> gh_plot_coh, freq, coh, coh_err
```

If the reference band includes the subject band, a real-part correction is especially important because overlapping photons introduce partial correlation. In that case `gh_cs` with `POISSON=[f1,f2]` is the preferred starting point.

Chapter 8

Dynamic Power Spectra

A dynamic power spectrum, or spectrogram, shows how the PDS changes with time. In GHATS this is not made directly from an event file. The event file is first converted into a normal GHATS `.pds` product, and the dynamic-PDS routines then read the individual spectra stored in that file.

The basic workflow is:

```
Ghats> gh_nicer, 'event.evt', 'POWER', [30,1200], 10000, 32768, 'event.pds'
Ghats> gh_dyn, 'event.pds', time, rate, frequency, dynimage
```

or, if a FITS file and viewer are wanted:

```
Ghats> gh_dyn_fits, 'event.pds', 'event_dynpds.fits', frebin=-100, trebin=4, /show
```

8.1 What Is Stored in a Dynamic PDS

`gh_dyn` reads the powers already stored in a GHATS `.pds` file. Each transform becomes one row in the time-frequency image before optional rebinning. The output arrays are:

`time` Time axis, in seconds from the file start, after optional time rebinning.

`rate` Count-rate curve, rebinned in the same time bins as the image.

`frequency` Frequency axis, in Hz, after optional frequency rebinning.

`dynimage` Two-dimensional power image. The first dimension is time and the second dimension is frequency.

The powers are the powers stored in the input `.pds` file. The routine does not recompute Fourier transforms and does not subtract a Poisson level. If a noise-subtracted or differently normalized view is needed, that step has to be handled outside `gh_dyn`.

8.2 `gh_dyn`

The simplest call is:

```
Ghats> gh_dyn, 'file.pds', time, rate, frequency, dynimage
```

The main optional keywords are:

`INDEX=[i1,i2]` Select a transform-index range.

FREBIN=fr Rebin the frequency axis. Positive values request linear rebinning. Negative values request logarithmic rebinning, with `-100` a common value for inspection plots. The default is 1, meaning no frequency rebinning.

TREBIN=tr Rebin the time axis linearly by this factor. Only linear time rebinning is supported. The default is 1.

/HELP Print the routine help screen.

For example:

```
Ghats> gh_dyn, 'file.pds', time, rate, frequency, dynimage, $
       index=[0,199], frebin=-100, trebin=4
```

This reads the selected part of the PDS file, logarithmically rebins each PDS in frequency, and combines groups of four adjacent time bins.

> **Note.** The dynamic image is intended as a visual and exploratory product. If an average PDS with errors is needed, use `ghx` and `gh_reb` rather than averaging rows of `dynimage` by hand.

8.3 Frequency Rebinning

`FREBIN` uses the same sign convention as other GHATS rebinning routines:

FREBIN=1 Keep the native frequency grid.

FREBIN=n, $n > 1$ Combine neighbouring frequency bins linearly.

FREBIN=-n Rebin logarithmically, with bin widths increasing by approximately $\exp(1/n)$.

The frequency rebinning is applied to every PDS row independently. The returned `frequency` array is the common rebinned frequency axis.

8.4 Time Rebinning

`TREBIN` combines adjacent spectra in time. A value of 4, for example, groups four consecutive PDS rows into one output time bin:

```
Ghats> gh_dyn, 'file.pds', time, rate, frequency, dynimage, trebin=4
```

The light curve returned in `rate` is rebinned consistently with the dynamic image. Time rebinning is linear only; logarithmic time rebinning is not implemented.

8.5 gh_dyn_fits

`gh_dyn_fits` is a convenience wrapper. It runs `gh_dyn` and writes the result to a FITS file:

```
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits'
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits', $
       index=[0,199], frebin=-100, trebin=4
```

The same `INDEX`, `FREBIN`, and `TREBIN` keywords are passed to `gh_dyn`. With `/SHOW`, the routine launches the Python viewer after the FITS file is written:

```
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits', frebin=-100, /show
```

Additional viewer options can be passed as a string through `ARGUMENTS`:

```
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits', /show, $
       arguments='--linear -fmin 0.1 -fmax 50 -pmin 1.5 -pmax 4.0'
```

For the list of colour maps accepted by `-cmap`, use:

```
Ghats> gh_dyn_fits, /cmhelp
```

Any listed colour map can be reversed by appending `_r`, for example `viridis\_r`.

8.6 Dynamic-PDS FITS Layout

The current FITS file written by `gh_dyn_fits` is simple and uses HDU order rather than named extensions:

HDU	Contents	Meaning
Primary	`dynima`	Dynamic PDS image.
Extension 1	`nu`	Frequency array in Hz.
Extension 2	`time`	Time array in seconds from the file start.
Extension 3	`rate`	Count-rate array.

Note. The current dynamic-PDS FITS writer does not add custom `EXTNAME` keywords or detailed axis metadata. Readers should therefore treat the HDU order above as part of the current file format.

8.7 Python Viewer: `GH_dyn_fits_tk.py`

`GH_dyn_fits_tk.py` reads the FITS files written by `gh_dyn_fits` and opens an interactive viewer:

```
$ GH_dyn_fits_tk.py file_dynpds.fits
$ GH_dyn_fits_tk.py file_dynpds.fits -fmin 0.1 -fmax 50 -pmin 1.5 -pmax 4.0
```

The script lives in the repository `PYTHON` directory. To run it by name from the shell, or through `gh_dyn_fits` with `/SHOW`, the `PYTHON` directory should be in the shell path used by the GHATS session.

Common viewer options include:

`-title` Set the window title.

`-t0` Set the reference MJD for the time axis.

`-cmap` Choose a Matplotlib colour map. The `/CMHELP` screen lists the available names from inside GHATS.

`-interp` Choose the image interpolation method.

`-pmin`, `-pmax` Set the colour-scale limits.

`-fmin`, `-fmax` Set the displayed frequency range.

-ft Set frequency tick positions.

-fh Highlight selected frequencies.

-hc Set the highlight colour.

-tmin, -tmax Set the displayed time range.

-tt Set time tick positions.

-th Highlight selected times.

--linear Use a linear colour scale instead of the default logarithmic display.

--nupnu Display $fP(f)$ rather than $P(f)$.

-cb, --colorbar Show the colour bar at startup.

--dynPDSonly Hide the light-curve panel.

--savepdf, --savepng Save the figure.

--noshow Save or prepare the figure without displaying the interactive window.

--showGaps, --fillGaps Mark or fill gaps in the time axis when the viewer can infer them.

The full list is available from:

```
$ GH_dyn_fits_tk.py -h
```

8.8 A Compact Dynamic-PDS Workflow

A typical sequence is:

```
Ghats> gh_info, 'file.pds'
Ghats> gh_dyn, 'file.pds', time, rate, frequency, dynimage, $
       frebin=-100, trebin=4
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits', $
       frebin=-100, trebin=4
```

For interactive inspection:

```
Ghats> gh_dyn_fits, 'file.pds', 'file_dynpds.fits', $
       frebin=-100, trebin=4, /show, $
       arguments='-fmin 0.1 -fmax 64 --showGaps'
```

This keeps the GHATS data product and the visualisation step separate: the `.pds` file remains the authoritative timing product, while the FITS file is a portable image product for inspection and plotting.

Chapter 9

XSPEC Products

GHATS can export one-dimensional timing spectra to files that can be read by XSPEC or ISIS. The exported product is not an X-ray energy spectrum. It is a timing spectrum written in OGIP-style PHA/RMF form so that XSPEC can be used as a convenient fitting engine.

The usual entry point is `gh_xspec`. It takes arrays already in memory, writes a `.pha` file containing the data and errors, and writes a matching diagonal `.rmf` response whose channels represent frequency bins.

> **Note.** In the XSPEC files written by GHATS, the "energy" axis is the Fourier frequency axis. The numerical channel boundaries are frequency-bin boundaries, normally in Hz. The files are intended for fitting timing products, not for spectral-response analysis.

9.1 `gh_xspec`

`gh_xspec` writes a timing spectrum to XSPEC format:

```
Ghats> gh_xspec, frequency, value, value_err, 'output_name'
```

This creates

- `output_name.pha`, the PHA file containing the data values and statistical errors;
- `output_name.rmf`, the diagonal response file defining the frequency bins.

The arrays are:

`frequency` Central frequency of each bin.

`value` The timing quantity to be fitted.

`value_err` The statistical error on `value`.

`output_name` Basename for the output files. Do not include the `.pha` or `.rmf` extension.

The input `value` array is assumed to be a density-like quantity on a frequency grid, as returned by GHATS analysis routines. Before writing the PHA file, `gh_xspec` multiplies each value and error by twice the corresponding half-bin width:

$$\mathrm{PHA}_i = \mathrm{value}_i \, 2\Delta f_i, \qquad \sigma_{\mathrm{PHA},i} = \sigma_{\mathrm{value},i} \, 2\Delta f_i.$$

The frequency-bin boundaries written to the response are

$$f_{1,i} = f_i - \Delta f_i, \qquad f_{2,i} = f_i + \Delta f_i.$$

For the first bin, Δf is half the separation between the first two input frequencies. For later bins, the routine reconstructs the half-bin width from the spacing of the frequency array. This means that the input `frequency` array should be ordered and should describe the actual bin centres of the data being exported.

9.1.1 Common Examples

The routine can be used after reading and rebinning a PDS:

```
Ghats> ghx, 'event.pds', freq, power, power_err
Ghats> gh_reb, freq, power, power_err, -100, rfreq, rpower, rpower_err
Ghats> gh_xspec, rfreq, rpower, rpower_err, 'event_rebinned'
```

It can also be used for cross-spectral products returned by `gh_cs`. For example:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf

Ghats> gh_xspec, freq, real, real_err, 'real_ref_sub'
Ghats> gh_xspec, freq, imag, imag_err, 'imag_ref_sub'
Ghats> gh_xspec, freq, lag,  lag_err,  'plag_ref_sub'
Ghats> gh_xspec, freq, coh,  coh_err,  'cohe_ref_sub'
Ghats> gh_xspec, freq, modcs, modcs_err, 'mod_ref_sub'
```

The names are arbitrary, but using descriptive prefixes such as `real`, `imag`, `plag`, `cohe`, and `mod` helps keep the XSPEC session understandable.

9.1.2 Keywords

`/XSPEC` After writing the files, launch XSPEC and load the new PHA file. The routine also writes a small `.xcm` command file. This option is not available under Windows.

`TELESCOPE=tel` Telescope name written to the FITS headers.

`INSTRUMENT=inst` Instrument name written to the FITS headers.

`EXTRA_KEYS=keys, EXTRA_VALUES=values` Additional header keywords and values passed to the PHA writer. The two arrays must have the same number of elements.

For example:

```
Ghats> gh_xspec, freq, power, power_err, 'event_rebinned', $
       telescope='XTE', instrument='PCA'
```

9.1.3 Bad Channels

If `value` or `value_err` contains undefined values, the corresponding XSPEC channel is marked bad through the PHA `QUALITY` column. This is useful when arrays produced by cross-spectral routines contain bins that should not be used in a fit, for example low-coherence bins that were flagged as undefined.

The data are still written to disk. In XSPEC, the usual channel-quality machinery can then be used to ignore bad bins.

9.2 gh_pha and gh_rmf

`gh_pha` and `gh_rmf` are the lower-level writers used by `gh_xspec`. They are normally not called directly by the user.

9.2.1 gh_pha

`gh_pha` writes the PHA file:

```
Ghats> gh_pha, power, power_err, quality, phaname, rmfname, $
       telescope, instrument
```

The file contains a primary FITS header and a binary table extension named `SPECTRUM`. The main columns are:

Column	Meaning
`CHANNEL`	XSPEC channel number, starting from 1.
`COUNTS`	Timing value converted to power per bin by `gh_xspec`.
`STAT_ERR`	Statistical error on `COUNTS`.
`QUALITY`	Channel-quality flag. A value of 0 marks a good channel; non-zero values mark channels that should be treated as bad.

The PHA header points to the associated response file through `RESPFILE`. The exposure, area scaling, background scaling, and correction scaling are set to unity, and `POISSERR` is set to false because GHATS provides the errors explicitly.

9.2.2 gh_rmf

`gh_rmf` writes the diagonal response:

```
Ghats> gh_rmf, f_low, f_high, rmfname, telescope, instrument
```

The response contains two binary-table extensions:

`MATRIX` The response matrix. It is diagonal: each frequency bin maps to itself with matrix value 1.

`EBOUNDS` The lower and upper boundaries of each channel. In GHATS XSPEC products these are the frequency-bin boundaries.

The RMF is therefore only a bookkeeping device that lets XSPEC assign a coordinate to each timing bin. It does not describe an instrumental energy response.

9.3 Practical XSPEC Use

Once the files are written, they can be loaded in XSPEC in the usual way:

```
XSPEC> data event_rebinned.pha
XSPEC> setplot energy
XSPEC> plot ldata
```

The command `setplot energy` is useful because XSPEC uses the response coordinate as the plotting axis. In these files, that coordinate is frequency.

When several related timing quantities are written separately, for example $\mathrm{Re}(C)$, $\mathrm{Im}(C)$, phase lag, and coherence, each one is a separate PHA/RMF pair. Any simultaneous fit or comparison in XSPEC must be set up explicitly in the XSPEC session or command file.

Note. `gh_xspec` preserves the numerical values and errors provided by the GHATS analysis step, apart from the required conversion from density per Hz to per-bin PHA values. It does not rebin the data, subtract Poisson noise, or change GHATS Fourier, lag, or normalization conventions.

Chapter 10

Routine Reference

This chapter gives a compact directory of the main user-facing routines. It should be kept consistent with `gh_help` and each routine's `/HELP` output.

10.1 Directory of Routines

This directory is the first place to look for the name of a routine. It gives one line per command, grouped by task. Once the relevant command has been identified, run it with `/HELP` for detailed syntax:

```
Ghats> gh_cs, /help
```

The same directory can be opened inside GHATS with:

```
Ghats> gh_help, /task
Ghats> gh_help, /alpha
```

10.1.1 Start Here

Routine	Area	Purpose
`gh_help`	HELP	Open this routine directory in a pager.
`gh_version`	ANALYSIS	Print the GHATS version string.
`gh_info`	ANALYSIS	Print basic header information for a GHATS PDS or FFT file.
`ghats_all`	ANALYSIS	Produce a quick summary plot/report from a PDS file.

10.1.2 Produce PDS and FFT Files

Routine	Area	Purpose
`gh_xte`	FFT	Produce GHATS PDS/FFT files from RXTE/PCA data.
`gh_nicer`	NICER	Produce GHATS PDS/FFT files from NICER event data.
`gh_ep`	Einstein Probe	Produce GHATS PDS/FFT files from Einstein Probe event data.

Routine	Area	Purpose
gh_nustar	NuSTAR	Produce GHATS PDS/FFT files from NuSTAR event data.
gh_xmm	XMM	Produce GHATS PDS/FFT files from XMM event data.
gh_swift	Swift	Produce GHATS PDS/FFT files from Swift/XRT event data.
gh_ixpe	IXPE	Produce GHATS PDS/FFT files from IXPE event data.
gh_laxpc	AstroSat/LAXPC	Produce GHATS PDS/FFT files from AstroSat/LAXPC data.
gh_laxpc_misra	AstroSat/LAXPC	Produce GHATS PDS/FFT files from AstroSat/LAXPC event data.
gh_czti	AstroSat/CZTI	Produce GHATS PDS/FFT files from AstroSat/CZTI event data.
gh_huiyan	Huiyan	Produce GHATS PDS/FFT files from Huiyan/HXMT data.
gh_huiyan_le	Huiyan/LE	Produce GHATS PDS/FFT files from Huiyan/HXMT LE data.
gh_huiyan_me	Huiyan/ME	Produce GHATS PDS/FFT files from Huiyan/HXMT ME data.
gh_huiyan_he	Huiyan/HE	Produce GHATS PDS/FFT files from Huiyan/HXMT HE data.
gh_ascii_lcfft	CONTRIBUTED	Produce GHATS FFT products from ASCII light curves.

10.1.3 Read, Average, and Rebin PDS/FFT Products

Routine	Area	Purpose
ghx	ANALYSIS	Read and average GHATS PDS files.
ghx_fft	ANALYSIS	Read and average GHATS FFT products.
ghx_poisrange	CONTRIBUTED	Read PDS files with Poisson level estimated from a frequency range.
gh_reb	ANALYSIS	Rebin a PDS linearly or logarithmically.
gh_licu	ANALYSIS	Extract a light curve from a GHATS PDS or FFT file.
gh_nspectra	ANALYSIS	Read the number of spectra/transforms in a GHATS PDS or FFT file.
gh_colors	ANALYSIS	Extract X-ray colors from a GHATS PDS file.
gh_dyn	ANALYSIS	Extract a dynamic PDS/time-frequency image from a PDS file.
gh_dyn_fits	ANALYSIS	Save a dynamic PDS/time-frequency image to FITS.

Routine	Area	Purpose
gh_getvle	ANALYSIS	Extract VLE information from a PDS file.
gh_hk_xte	ANALYSIS	Extract RXTE/PCA housekeeping information from PDS/FFT products.

10.1.4 Cross Spectra, Lags, and Coherence

Routine	Area	Purpose
gh_cs	CROSS	Compute lags, coherence, Re/Im, and modulus with cross-talk correction.
gh_cross_re_im_new	CROSS	Compute cross-spectrum lags, coherence, Re/Im, and modulus from two FFT files.
gh_cross	ANALYSIS	Compute phase/time lags and coherence from two compatible FFT files.
gh_cross_range	ANALYSIS	Compute cross-spectrum quantities around a selected frequency range.
gh_plot_lag	ANALYSIS	Plot phase or time lags with errors.
gh_plot_coh	ANALYSIS	Plot coherence with errors.

10.1.5 Bispectrum and Bicoherence

Routine	Area	Purpose
gh_bispec	BISPECTRUM	Compute 1D/target bispectrum products from a GHATS FFT file.
gh_bispec2d	BISPECTRUM	Compute 2D bispectrum and squared bicoherence from a GHATS FFT file.
gh_bispec2d_rebin	BISPECTRUM	Rebin raw 2D bispectrum sums onto coarser frequency grids.
gh_bispec_fits	BISPECTRUM	Compute 2D bispectrum products and write them to FITS.
gh_bispec_plot	BISPECTRUM	Plot 2D bispectrum, biphase, bicoherence, and related maps.
gh_bispec_hypot	BISPECTRUM	Extract or summarize bispectrum values along hypotenuse-like regions.
gh_bispec_freqband_pair_matrix	BISPECTRUM	Summarize bicoherence/biphase between frequency bands.

Routine	Area	Purpose
gh_bispec_lorentz_pair_matrix	BISPECTRUM	Summarize bicoherence/biphase between Lorentzian component bands.
gh_bispec_target_pair_matrix	BISPECTRUM	Summarize target-band bispectrum coupling between frequency bands.
gh_cross_bispec2d	BISPECTRUM	Compute 2D cross-bispectrum products from GHATS FFT files.
gh_cross_bispec2d_match	BISPECTRUM	Match or compare 2D cross-bispectrum products.
gh_cross_bispec_target_segments	BISPECTRUM	Extract target cross-bispectrum values by segment.
gh_write_bispec_fits	BISPECTRUM	Write auto-bispectrum products to FITS.
gh_write_cross_bispec_fits	BISPECTRUM	Write cross-bispectrum products to FITS.
gh_read_bispec_fits	BISPECTRUM	Read bispectrum products from FITS.

10.1.6 Plot PDS, Light Curves, and Diagnostics

Routine	Area	Purpose
gh_plot_power	ANALYSIS	Plot a PDS in $P(f)$ form.
gh_plot_nupower	ANALYSIS	Plot a PDS in $fP(f)$ form.
gh_oplot_power	ANALYSIS	Overplot a PDS in $P(f)$ form.
gh_oplot_nupower	ANALYSIS	Overplot a PDS in $fP(f)$ form.
gh_plot_licu	ANALYSIS	Plot a light curve extracted from a PDS/FFT product.
gh_plot_licu_nogaps	ANALYSIS	Plot a light curve while suppressing gaps.
gh_plot_hk_xte	ANALYSIS	Plot RXTE/PCA housekeeping curves.

10.1.7 XSPEC and FITS Output

Routine	Area	Purpose
gh_xspec	ANALYSIS	Write arrays as XSPEC PHA/RMF products.
gh_pha	ANALYSIS	Write a PHA file for XSPEC; normally called by gh_xspec.

Routine	Area	Purpose
`gh_rmf`	ANALYSIS	Write a diagonal RMF/RSP file for XSPEC; normally called by `gh_xspec`.
`gh_dyn_fits`	ANALYSIS	Save dynamic PDS products to FITS.
`gh_bispec_fits`	BISPECTRUM	Write bispectrum products to FITS.
`gh_write_bispec_fits`	BISPECTRUM	Write auto-bispectrum products to FITS.
`gh_write_cross_bispec_fits`	BISPECTRUM	Write cross-bispectrum products to FITS.

10.1.8 File Utilities and Low-Level Readers

Routine	Area	Purpose
`ghats_openpds`	ANALYSIS	Open a GHATS PDS file; low-level helper.
`ghats_openfft`	ANALYSIS	Open a GHATS FFT file; low-level helper.
`ghats_getheader`	ANALYSIS	Read a GHATS PDS/FFT header; low-level helper.
`read_pds_line`	ANALYSIS	Read the next PDS record from an open GHATS PDS file.
`read_fft_line`	ANALYSIS	Read the next FFT record from an open GHATS FFT file.
`ghx_read_pds_metafile`	ANALYSIS	Read and validate a `ghx` PDS metafile.
`gh_split_pds`	CONTRIBUTED	Split GHATS PDS products into smaller files.
`gh_split_fft`	CONTRIBUTED	Split GHATS FFT products into smaller files.

Chapter 11

Walkthroughs

This chapter gives worked examples for common analyses.

11.1 Basic PDS Analysis

This walkthrough starts from an existing GHATS `.pds` file and follows the usual quick-look path: inspect the file, read and average the power spectra, subtract a noise level if appropriate, rebin the result, and make a plot. The same sequence can be used for products produced by any of the mission-specific routines described in Chapter 4.

Inspect the File

Before averaging or plotting, check that the file is the one intended for the analysis:

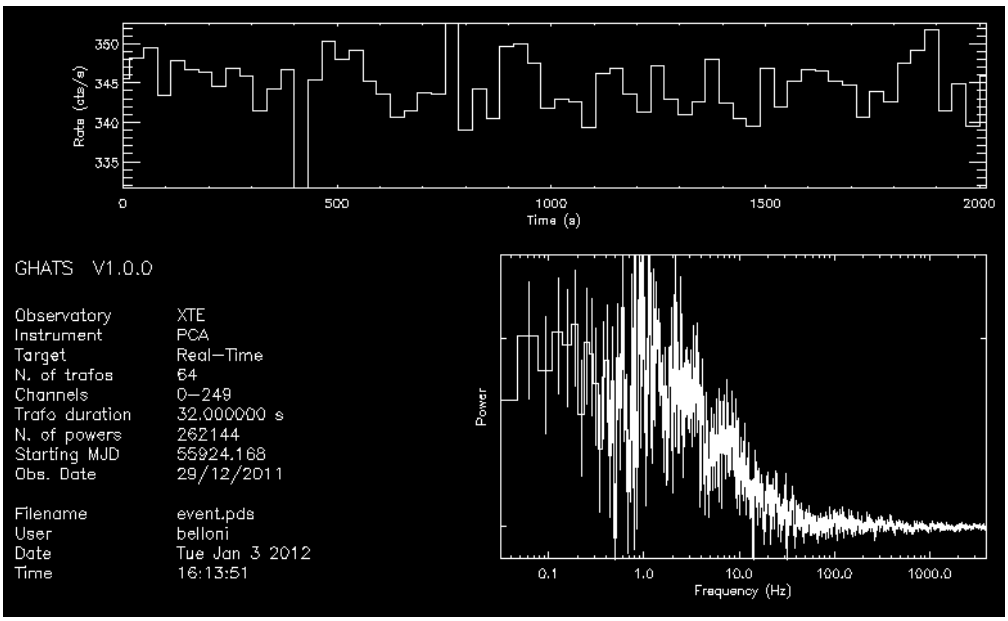
```
Ghats> gh_info, 'source.pds'
```

`gh_info` prints the observatory, instrument, target, number of stored transforms, number of time bins per transform, frequency resolution, Nyquist frequency, channel range, and start time. These values are the basic sanity checks for a PDS analysis. In particular, the frequency resolution and Nyquist frequency tell the user whether the chosen transform length and time binning are suitable for the variability range of interest.

For a more visual first look, use:

```
Ghats> ghats_all, 'source.pds'
```

`ghats_all` prints a similar header summary, extracts the stored light curve, averages the PDS, and makes a quick diagnostic plot. It is a convenience routine for inspection; for measurements, read the arrays explicitly as shown below.



Example `ghats_all` summary plot from the original GHATS manual. The upper panel shows the count rate for each PDS interval, and the lower panel shows the averaged PDS.

Read and Average the PDS

The basic call to `ghx` is:

```
Ghats> ghx, 'source.pds', freq, pow, powe
```

This selects all transforms in the file and returns the average PDS in `pow`, with the corresponding frequency array in `freq` and error array in `powe`. If no noise option is supplied, `ghx` returns the raw PDS and prints a warning to make that explicit.

To select only part of the observation, use the standard selection keywords:

```
Ghats> ghx, 'source.pds', freq, pow, powe, time=[0.0,1000.0]
Ghats> ghx, 'source.pds', freq, pow, powe, index=[0,63]
Ghats> ghx, 'source.pds', freq, pow, powe, rate=[100.0,500.0]
```

The selections may be combined. The `INDEX` values refer to the internal transform counter, starting from 0.

Subtract the Noise Level

For RXTE/PCA products, the historical option is:

```
Ghats> ghx, 'source.pds', freq, pow, powe, /poisson
```

For a mission-independent empirical estimate, give a frequency interval from which `ghx` should estimate a constant level:

```
Ghats> ghx, 'source.pds', freq, pow, powe, poisson=[400.0,800.0]
```

The interval is clipped to the available non-Nyquist frequencies. If a constant level has already been measured elsewhere, it can be supplied directly:

```
Ghats> ghx, 'source.pds', freq, pow, powe, manual_poi=2.0
```

Use only one of `/POISSON`, `POISSON=[f1,f2]`, or `MANUAL_POI=p` in a single call.

Convert to Fractional-RMS Units

If the PDS should be returned in rms^2 units, supply the background count rate through `RMS`:

```
Ghats> ghx, 'source.pds', freq, pow, powe, poisson=[400.0,800.0], rms=0.0
```

The value passed with `RMS` is the background rate in the same count-rate units as the GHATS product. For a negligible or unknown background, users often start with `rms=0.0`; if the background is important, it should be estimated from the appropriate mission data products and supplied here.

Rebin the PDS

The raw frequency grid is often too fine for display. A common next step is logarithmic rebinning with a negative rebin factor:

```
Ghats> gh_reb, freq, pow, powe, -100, rfreq, rpow, rpowe
```

The output arrays `rfreq`, `rpow`, and `rpowe` contain the rebinned spectrum. Larger absolute values of a negative rebin factor give finer logarithmic sampling; smaller absolute values give stronger smoothing. For linearly grouped bins, use a positive integer rebin factor instead.

Plot the Result

To plot $P(f)$:

```
Ghats> gh_plot_power, rfreq, rpow, rpowe
```

To plot $fP(f)$:

```
Ghats> gh_plot_nupower, rfreq, rpow, rpowe
```

Optional axis limits can be supplied after the arrays:

```
Ghats> gh_plot_power, rfreq, rpow, rpowe, 0.1, 64.0
Ghats> gh_plot_power, rfreq, rpow, rpowe, 0.1, 64.0, 1.0e-4, 10.0
```

The first pair is the frequency range. The second pair, when present, is the power range.

A Complete Minimal Session

A compact first-pass analysis might therefore look like this:

```
Ghats> gh_info, 'source.pds'
Ghats> ghx, 'source.pds', freq, pow, powe, poisson=[400.0,800.0], rms=0.0
Ghats> gh_reb, freq, pow, powe, -100, rfreq, rpow, rpowe
Ghats> gh_plot_nupower, rfreq, rpow, rpowe
```

The arrays remain available in the IDL/GDL session after the plot. They can be fitted, written out, overplotted with another observation, or passed to other GHATS routines.

11.2 Cross-Spectral Analysis

This walkthrough starts from two GHATS `.fft` files and produces phase lags, coherence, and the real and imaginary parts of the cross spectrum. The two FFT files should normally be produced from the same event data with the same time binning, GTIs, transform length, and window function, changing only the channel or energy selection.

Create Compatible FFT Files

For example, for a soft and hard band:

```
Ghats> gh_nicer, 'event.evt', 'FFT', [30,200],  10000, 32768, 'soft.fft'
Ghats> gh_nicer, 'event.evt', 'FFT', [201,1200],10000, 32768, 'hard.fft'
```

For another mission, replace `gh_nicer` with the appropriate production routine. The important point is that the two calls use the same input data and the same timing parameters. The FFT products can be checked with:

```
Ghats> gh_info, 'soft.fft'
Ghats> gh_info, 'hard.fft'
```

The number of transforms, transform duration, frequency resolution, Nyquist frequency, and start time should match. If they do not, the files should not be used together for cross-spectral work.

Run the Recommended Cross-Spectral Command

The recommended high-level command is `gh_cs`:

```
Ghats> file_ref = 'soft.fft'
Ghats> file_sub = 'hard.fft'
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400.0,800.0], /lowcoh
```

The first file is the reference band and the second file is the subject band. Positive lags mean that the second band lags the first. The rebin factor `-100` requests the usual logarithmic frequency rebinning.

The output arrays are:

`freq` Rebinned frequency array, in Hz.

`lag`, `lag_err` Phase lag and error, in radians.

`coh`, `coh_err` Coherence and error.

`real`, `imag` Real and imaginary parts of the averaged cross spectrum.

`real_err`, `imag_err` Errors on the real and imaginary parts.

`modcs`, `modcs_err` Modulus of the cross spectrum and its error.

`mw` Number of averaged cross vectors in each rebinned bin.

`qf` Quality flag for the `/LOWCOH` coherence-error calculation.

`POISSON=[400.0,800.0]` tells the routine to estimate the PDS noise levels from that frequency interval. In `gh_cs`, the same interval is also used to estimate and subtract the constant real cross-spectrum component caused by dead-time cross talk or by overlap between subject and reference bands. The interval should lie inside the available frequency range and should not rely on the Nyquist bin.

Raw and Intrinsic Coherence

For intrinsic coherence, give either `POISSON=[f1,f2]` or `MANUAL_POI=[p1,p2]`:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       manual_poi=[2.0,2.0]
```

For raw measured coherence, use `/RAWCOH`. In that case the observed PDS are used in the denominator and no Poisson correction is applied:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, /rawcoh
```

`/RAWCOH` is mutually exclusive with `POISSON`, `MANUAL_POI`, and `/LOWCOH`.

Low-Coherence Errors and Quality Flags

With `/LOWCOH`, `gh_cs` uses the implemented Vaughan–Nowak low-measured-coherence error treatment where the conditions allow it. The quality flag `qf` records the path used for each frequency bin. Some bins may be outside the implemented useful range; in that case the routine marks them as undefined.

In IDL this may print:

```
% Program caused arithmetic error: Floating illegal operand
```

When it appears immediately after a `/LOWCOH` calculation, this message is normally the result of marking those undefined bins. Other floating-point messages should be checked.

RMS Units and Rotation

If `RMS1` and `RMS2` are supplied, the real and imaginary parts, their errors, and the modulus are converted to rms units:

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400.0,800.0], rms1=0.001, rms2=0.001, /lowcoh
```

The two values are the background rates for the first and second FFT files. They must be supplied together.

The `/ROTATE` option rotates the returned real and imaginary parts by $\pi/4$. This can be useful when the real part is large and the imaginary part is small or negative, for example before plotting both components in a logarithmic package. The lag and coherence arrays are not rotated.

```
Ghats> gh_cs, file_ref, file_sub, freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400.0,800.0], /lowcoh, /rotate
```

Plot or Export the Results

The lag and coherence arrays can be plotted directly:

```
Ghats> gh_plot_lag, freq, lag, lag_err, 0.1, 64.0, -1.0, 1.0, /lin
Ghats> gh_plot_coh, freq, coh, coh_err, 0.1, 64.0, 0.0, 1.1
```

For XSPEC fitting or plotting, write each array as a PHA/RMF pair:

```
Ghats> gh_xspec, freq, real, real_err, 'real_soft_hard'
Ghats> gh_xspec, freq, imag, imag_err, 'imag_soft_hard'
Ghats> gh_xspec, freq, lag,  lag_err,  'plag_soft_hard'
Ghats> gh_xspec, freq, coh,  coh_err,  'cohe_soft_hard'
```

A Complete Minimal Session

```
Ghats> gh_info, 'soft.fft'
Ghats> gh_info, 'hard.fft'
Ghats> gh_cs, 'soft.fft', 'hard.fft', freq, lag, lag_err, coh, coh_err, $
       -100, real, real_err, imag, imag_err, modcs, modcs_err, mw, qf, $
       poisson=[400.0,800.0], /lowcoh
Ghats> gh_plot_lag, freq, lag, lag_err, 0.1, 64.0, -1.0, 1.0, /lin
Ghats> gh_plot_coh, freq, coh, coh_err, 0.1, 64.0, 0.0, 1.1
```

11.3 Dynamic PDS Analysis

This walkthrough starts from an existing GHATS `.pds` file and constructs a time-frequency image. The individual rows of the image are the power spectra already stored in the PDS file; no new Fourier transforms are computed.

Start from a PDS File

First inspect the file:

```
Ghats> gh_info, 'source.pds'
```

The transform duration sets the time sampling of the unrebinned dynamic PDS. The frequency resolution and Nyquist frequency set the native frequency grid. If the dynamic PDS should show faster or slower changes, the PDS file itself must be produced with an appropriate transform length.

Read a Dynamic PDS into IDL/GDL Arrays

The simplest call is:

```
Ghats> gh_dyn, 'source.pds', time, rate, freq, dynimage
```

The returned arrays are:

`time` Time axis, in seconds from the file start.

`rate` Count-rate curve rebinned consistently with the image.

`freq` Frequency axis, in Hz.

`dynimage` Two-dimensional power image.

For inspection, it is usually useful to rebin in frequency and sometimes in time:

```
Ghats> gh_dyn, 'source.pds', time, rate, freq, dynimage, frebin=-100, trebin=4
```

`FREBIN=-100` uses logarithmic frequency rebinning. A positive `FREBIN` value would combine that number of adjacent frequency bins linearly. `TREBIN=4` combines groups of four adjacent PDS intervals in time. Time rebinning is linear only.

> **Note.** `gh_dyn` does not subtract a Poisson level. The dynamic image is primarily an inspection product. For an averaged PDS with errors, use `ghx` and `gh_reb`.

Write a FITS Dynamic PDS

To save the dynamic PDS as a FITS file, use `gh_dyn_fits`:

```
Ghats> gh_dyn_fits, 'source.pds', 'source_dynpds.fits', $
       frebin=-100, trebin=4
```

The FITS file contains the image, frequency array, time array, and count-rate array. It is a portable product for later inspection or plotting.

Open the Interactive Viewer

With `/SHOW`, `gh_dyn_fits` writes the FITS file and then launches the Python viewer:

```
Ghats> gh_dyn_fits, 'source.pds', 'source_dynpds.fits', $
       frebin=-100, trebin=4, /show
```

Viewer options can be passed through `ARGUMENTS`:

```
Ghats> gh_dyn_fits, 'source.pds', 'source_dynpds.fits', $
       frebin=-100, trebin=4, /show, $
       arguments='-fmin 0.1 -fmax 64 --showGaps'
```

The same viewer can also be run from the shell if the repository `PYTHON` directory is in the shell path:

```
$ GH_dyn_fits_tk.py source_dynpds.fits -fmin 0.1 -fmax 64 --showGaps
```

Useful viewer options include `-pmin` and `-pmax` for the colour scale, `-fmin` and `-fmax` for the frequency range, `-tmin` and `-tmax` for the time range, `-ft` and `-tt` for custom frequency and time tick positions, `-fh` and `-th` for highlighted frequencies and times, `--linear` for a linear colour scale, `--nupnu` for plotting $fP(f)$, `--dynPDSonly` for hiding the light-curve panel, and `-cb`/`--colorbar` for showing the colour bar at startup. To list the colour maps accepted by `-cmap`, run:

```
Ghats> gh_dyn_fits, /cmhelp
```

Any listed colour map can be reversed by appending `_r`.

A Complete Minimal Session

```
Ghats> gh_info, 'source.pds'
Ghats> gh_dyn, 'source.pds', time, rate, freq, dynimage, frebin=-100, trebin=4
Ghats> gh_dyn_fits, 'source.pds', 'source_dynpds.fits', $
       frebin=-100, trebin=4, /show, $
       arguments='-fmin 0.1 -fmax 64 --showGaps'
```

This leaves the original `.pds` file unchanged, creates a FITS time-frequency product, and opens a viewer for interactive inspection.

Appendix A

Window Functions

Here are listed the functional forms of the available window functions. The functions refer to the time interval $[-T/2, T/2]$ and are zero outside it. The parameters α, β, σ, and γ must be supplied by the user for the windows that require them.

In the implementation, `ghats_window` uses the dimensionless variable

$$u = t/T, \qquad -1/2 \le u < 1/2.$$

Window	Functional form	Parameter
Boxcar	$f(u) = 1$	none
Bartlett	$f(u) = 1 - \left\lvert \frac{u}{1/2} \right\rvert$	none
Hann	$f(u) = \frac{1}{2}\left[1 + \cos(2\pi u)\right]$	none
Welch	$f(u) = 1 - \left(\frac{u}{1/2}\right)^2$	none
Cosine	$f(u) = \cos(\pi u)$	none
Hanning	$f(u) = \cos^2(\pi u)$	none
Hamming	$f(u) = \alpha + (1-\alpha)\cos^2(\pi u)$	α
Triplet	$f(u) = \exp\left(-\frac{\beta}{\lvert u \rvert}\right)\cos^2(\pi u)$	β

Window	Functional form	Parameter
Gauss	$$f(u) = \exp\left[-\frac{1}{2}\frac{u^2}{\sigma^2}\right]$$	σ
Kaiser	$$f(u) = \frac{I_0\left(\gamma\sqrt{1-(2u)^2}\right)}{I_0(\gamma)}$$	γ

The numerical implementation is in `ghats_window`. The default window is `Boxcar`. The windows `Hamming`, `Triplet`, `Gauss`, and `Kaiser` require `WPAR`.

Appendix B

GHATS File Formats

This appendix describes the file formats written and read by the current GHATS routines for native timing products and bispectrum products. It is a description of the implementation, not a new definition of the science conventions.

The source of truth is the IDL code. In particular, the native PDS and FFT formats are written and read by:

- `mu_write_single_power`
- `mu_write_single_fft`
- `mu_compute_fft`
- `ghats_getheader`
- `ghats_openpds`
- `ghats_openfft`
- `read_pds_line`
- `read_fft_line`

The dynamic-PDS FITS format is written by `gh_dyn_fits`. The bispectrum FITS products are written and read by:

- `gh_bispec_fits`
- `gh_write_bispec_fits`
- `gh_write_cross_bispec_fits`
- `gh_read_bispec_fits`

B.1 Scope

This appendix covers:

- GHATS binary `.pds` files;
- GHATS binary `.fft` files;
- GHATS dynamic-PDS FITS files written by `gh_dyn_fits`;
- GHATS bispectrum FITS files.

It does not cover mission event files read as input, log files, PostScript plots, screen output, text parameter files, file lists, metafiles, or XSPEC/OGIP products such as `.pha` and `.rmf`.

B.2 IDL Binary Products

GHATS `.pds` and `.fft` files are IDL unformatted binary files written with `WRITEU` and read with `READU`. The files use the native binary layout of the platform that wrote them. Current GHATS products are normally little-endian, but external readers should verify the header fields rather than assuming that a product is valid.

Both formats have the same fixed header and the same per-transform metadata record. The final payload in each transform differs:

- `.pds` stores one real power array per transform;
- `.fft` stores one complex Fourier-amplitude array per transform.

B.2.1 Shared Header

The header is written once at the beginning of the file by `mu_write_single_power` or `mu_write_single_fft`. It is read by `ghats_getheader`.

Field	IDL type	Meaning
`version_id`	string, 16 bytes	GHATS/version identifier.
`osserv`	string, 16 bytes	Observatory name.
`strumento`	string, 16 bytes	Instrument name.
`sorgente`	string, 16 bytes	Source or target name.
`rmjd`	double	Start time, in the GHATS MJD/RMJD convention used by the writer.
`nft`	long	Number of input time bins in each transform.
`T`	double	Frequency resolution, equal to 1/transform duration.
`ntotal_ffts`	long	Number of transforms written to the file.
`canali`	int[2]	Selected channel range.
`proliferation`	int	Proliferation factor used by the writer.
`baryflag`	int	Barycentric-correction flag.
`n_spectral_bins`	int	Number of stored spectral channels; current writers set this to 3.
`background_flag`	int	Background flag; current writers set this to 0.
`dummy`	byte[100]	Reserved flags and mission-dependent metadata.

The number of transforms, `ntotal_ffts`, begins at byte offset 84 in the current binary layout. Several producers initially write a placeholder value and patch this field after the final number of transforms is known.

The first bytes of `dummy` have established meanings in current products:

Field	Meaning
`dummy[0]`	Bit-coded turbo/GTI information. Current writers use 1 for turbo and add 2 when the GTI flag is set.
`dummy[1]`	Window-function code.

Some readers also inspect later `dummy` bytes for mission-specific quantities. Those bytes should be treated as reserved unless a product-specific writer documents them.

B.2.2 Frequency Grid

For both `.pds` and `.fft`, the stored frequency bins are reconstructed as:

```
frequency = (FINDGEN(nft / 2) + 1.0) * T
```

The DC bin is not stored. The stored array contains bins 1 through `nft`/2, including the Nyquist bin.

The segment duration represented by one transform is:

```
duration = 1.0 / T
```

The time resolution of the original binned light curve is:

```
dt = duration / nft
```

The Nyquist frequency is:

```
nyquist = nft * T / 2.0
```

B.2.3 Per-Transform Record

After the header, each transform is written as one binary record with common scalar metadata followed by the transform payload.

Field	IDL type	Meaning
`rmjd`	double	Time assigned to this transform.
`cnts`	float	Total counts in the transform after the windowing step used by the writer.
`fndet`	float	Number of active detectors, stored as a float.
`a0`	float	DC-related normalization field. For PDS products this is normally 2|DC|. For FFT products written by `mu_write_single_fft`, this is currently written as 0.
`poisson`	float	Poisson level stored with the transform.
`current_vle_rate`	float	VLE rate or mission-equivalent housekeeping value.
`std21`	float	Standard-2 or mission-dependent auxiliary value.
`std22`	float	Standard-2 or mission-dependent auxiliary value.
`std23`	float	Standard-2 or mission-dependent auxiliary value.
payload	format-dependent	Power or complex FFT data.

The scalar metadata are read by `read_pds_line` and `read_fft_line`.

B.3 `.pds` Files

A `.pds` file stores a power-density spectrum for each selected transform.

Field	IDL type	Length	Meaning
`pwr`	float array	`nft`/2	Power values for frequency bins 1 through `nft`/2.

B.3.1 PDS Normalization

For normal GHATS PDS production, `mu_compute_fft` computes the Fourier transform and then writes powers as:

```
pwr = ABS(rdata[1:np/2])^2 * 2.0 / cnts
```

Here `cnts` is the total number of counts in the windowed input segment. This is Leahy normalization.

For RXTE/PCA products, the Poisson level may be computed with the Zhang et al. prescription through `mu_zhang`. For non-PCA products, the stored Poisson level is normally 2.0 unless the mission writer sets something else.

B.4 `.fft` Files

A `.fft` file stores the complex Fourier amplitudes for each selected transform.

Field	IDL type	Length	Meaning
`rdata`	complex array	`nft`/2	Complex Fourier amplitudes for frequency bins 1 through `nft`/2.

IDL complex values are stored as two 32-bit floating-point numbers per element: real part followed by imaginary part.

B.4.1 FFT Convention

The stored FFT array is produced in `mu_compute_fft` as:

```
rdata = FFT(rdata, 1, /OVERWRITE)
fftdata = rdata[1:np/2]
```

The DC bin is not stored. GHATS therefore stores the positive-frequency bins produced by IDL `FFT(input, 1)`.

The source comment in `mu_compute_fft` notes that the IDL call used here corresponds to the forward transform in the convention used by the original GHATS timing routines. External code that computes cross spectra or lags should match this implementation convention rather than changing the lag sign convention.

B.4.2 FFT Units

The `.fft` payload is the raw complex Fourier amplitude. It is not saved in Leahy, rms, or fractional-rms normalization. Normalizations are applied later by analysis routines that read the FFT products.

B.5 Dynamic-PDS FITS Files

`gh_dyn_fits` writes a dynamic PDS or time-frequency image using `WRITEFITS`. The current HDU order is:

HDU	Contents	Notes
Primary	`dynima`	Dynamic PDS image.
Extension 1	`nu`	Frequency array.
Extension 2	`time`	Time array.
Extension 3	`rate`	Rate array.

The routine does not currently add custom `EXTNAME` keywords or detailed axis metadata. Readers of these files should therefore treat the HDU order above as part of the current format.

B.6 Bispectrum FITS Files

The current GHATS bispectrum FITS format is image-extension based and is written by `gh_bispec_fits` through `gh_write_bispec_fits`. The primary HDU contains a dummy one-byte image. This is intentional: it avoids problems with FITS writers that do not handle a zero-length primary image reliably.

B.6.1 Primary Header

The primary header records product-level metadata. Common keywords include:

Keyword	Meaning
`CREATOR`	Writer routine, normally `GH_BISPEC_FITS`.
`CONTENT`	Product description, normally `2D bispectrum products`.
`PRODUCT`	Product label, such as `BISPEC` or a cross-bispectrum label.
`SRCFILE1`, `SRCFILE2`, `SRCFILE3`	Optional source files used to make the product.
`NFREQ1`, `NFREQ2`	Number of frequency bins on the two bispectrum axes.
`F1MIN`, `F1MAX`	First-axis frequency range.
`F2MIN`, `F2MAX`	Second-axis frequency range.
`DF1`, `DF2`	Frequency spacing on each axis.
`BUNITB`	Unit string for `BREAL`, `BIMAG`, and `BMOD`.
`RAWSUMS`	Flag indicating whether raw-sum extensions are present.
`INTBICOH`	Flag indicating whether intrinsic-observed bicoherence extensions are present.
`NEXTEND`	Number of extensions expected by the writer.
`REBIN1`, `REBIN2`	Rebin factors for the two frequency axes.
`AXLOG1`, `AXLOG2`	Flags indicating logarithmic axis construction.

For intrinsic-observed bicoherence products, the primary header may also include:

Keyword	Meaning
POIMETH	Poisson correction method.
POILEVEL	Poisson level used.
POIFMIN, POIFMAX	Frequency range used for the Poisson estimate, when applicable.
BICNUM	Numerator convention. Current intrinsic-observed products use UNCHANGED.
BICDEN	Denominator convention. Current intrinsic-observed products use POISSON_CORR.

Cross-bispectrum products may also include:

Keyword	Meaning
CORRMODE	Correction mode.
DIAGCORR	Diagonal correction convention.
DTCOVAR	Dead-time covariance convention.
NUMBIAS	Numerator bias convention.
BAND1, BAND2, BAND3	Energy-band labels.
BANDOVLP	Band-overlap description.
XORDER	Cross-bispectrum ordering convention.
FCONV	Frequency or lag-sign convention note.

History records from the writer are stored as FITS HISTORY cards.

B.6.2 Required Extensions

The standard bispectrum product contains these extensions, in this order:

EXTNAME	Type	Shape	Units
BREAL	image	[NFREQ1, NFREQ2]	BUNITB
BIMAG	image	[NFREQ1, NFREQ2]	BUNITB
BMOD	image	[NFREQ1, NFREQ2]	BUNITB
BPHASE	image	[NFREQ1, NFREQ2]	radians
BICOH	image	[NFREQ1, NFREQ2]	dimensionless
NPROD_USED	long image	[NFREQ1, NFREQ2]	count
FREQ1	double image	[NFREQ1]	Hz
FREQ2	double image	[NFREQ2]	Hz

The two-dimensional image extensions include frequency-axis metadata:

Keyword	Axis 1 meaning	Axis 2 meaning
CTYPE1, CTYPE2	FREQ1	FREQ2

Keyword	Axis 1 meaning	Axis 2 meaning
CUNIT1, CUNIT2	Hz	Hz
CRPIX1, CRPIX2	Reference pixel, normally 1	Reference pixel, normally 1
CRVAL1, CRVAL2	First frequency value	First frequency value
CDELT1, CDELT2	Frequency spacing	Frequency spacing

The image extensions also carry the rebinning keywords REBIN1, REBIN2, AXLOG1, and AXLOG2.

The unit stored in BUNITB depends on the requested bispectrum normalization:

Normalization	BUNITB
rms	rms^3
Leahy	Leahy^(3/2)
other/native	native

B.6.3 Optional Raw-Sum Extensions

If raw sums are present, the writer appends:

EXTNAME	Type	Shape	Meaning
RAW_BSUM_REAL	image	[NFREQ1, NFREQ2]	Real part of the raw bispectrum sum.
RAW_BSUM_IMAG	image	[NFREQ1, NFREQ2]	Imaginary part of the raw bispectrum sum.
RAW_DEN1	image	[NFREQ1, NFREQ2]	First raw denominator sum.
RAW_DEN2	image	[NFREQ1, NFREQ2]	Second raw denominator sum.

These extensions use the same frequency-axis metadata as the required two-dimensional image extensions.

B.6.4 Optional Intrinsic-Observed Extensions

If intrinsic-observed bicoherence products are present, the writer appends:

EXTNAME	Type	Shape	Meaning
INT_BICOH	image	[NFREQ1, NFREQ2]	Intrinsic-observed bicoherence estimate.
RAW_DEN1_CORR	image	[NFREQ1, NFREQ2]	Corrected first denominator sum.
RAW_DEN2_CORR	image	[NFREQ1, NFREQ2]	Corrected second denominator sum.

These extensions carry the Poisson-correction metadata used for the intrinsic-observed product.

B.6.5 Cross-Bispectrum Extensions

`gh_write_cross_bispec_fits` uses the same core format and may append additional extensions for cross-bispectrum diagnostics:

EXTNAME	Type	Shape	Meaning
`INT_BICOH_INDEP`	image	[NFREQ1, NFREQ2]	Independent-noise intrinsic bicoherence estimate.
`RAW_DEN1_CORR_INDEP`	image	[NFREQ1, NFREQ2]	Independent-noise corrected first denominator.
`RAW_DEN2_CORR_INDEP`	image	[NFREQ1, NFREQ2]	Independent-noise corrected second denominator.
`INTR_VALID`	byte image	[NFREQ1, NFREQ2]	Validity mask for intrinsic quantities.
`CORR_FLAGS`	long image	[NFREQ1, NFREQ2]	Correction flags.
`DIAG_FLAG`	byte image	[NFREQ1, NFREQ2]	Diagonal-correction flag.

These extensions include the same frequency-axis metadata as the main two-dimensional bispectrum images, plus correction-convention keywords when supplied by the caller.

B.6.6 Reading Bispectrum FITS Products

`gh_read_bispec_fits` reads current products primarily by `EXTNAME`. It also contains fallback behaviour for older files with less complete extension naming. New readers should prefer `EXTNAME` over extension number wherever possible.

B.7 Compatibility Notes

The binary `.pds` and `.fft` formats are compact and historically tied to IDL's native binary representation. The FITS products are more self-describing, especially the bispectrum products, where extensions are named and frequency-axis metadata are written.

When checking compatibility between `.pds` files, routines should compare at least:

- `nft`;
- frequency resolution `T`;
- transform duration $1/T$;
- Nyquist frequency `nft T`/2;
- channel range;
- observatory and instrument metadata, when relevant to the analysis;
- normalization and Poisson-handling assumptions implied by the producing routine.

For `.fft` files, compatibility checks should additionally ensure that later cross-spectral or lag calculations use the same GHATS Fourier and lag conventions.

Appendix C

Conversion Notes

The legacy manual was authored in Apple Pages and exported to Word and PDF. This LaTeX version was started so that the manual can be edited, reviewed, and kept under version control more easily. The conversion is not intended to change the scientific content of GHATS. It is a change of source format and maintenance workflow.

C.1 Source Material

The recommended sources for future manual updates are:

- the Word document as the main source of recoverable text;
- the PDF as the visual reference for layout and style;
- the current routines and `gh_help` output as the source of truth for command behavior.

> **Note.** Do not change scientific definitions, Fourier conventions, lag sign conventions, normalization conventions, or file formats during manual conversion. Any such change must be discussed separately.

C.2 Editing Rules

When updating the LaTeX manual, edit one chapter or appendix at a time whenever possible. This keeps the review small and makes it easier to compare the new text with the old manual and with the routines it describes.

Routine names, filenames, keywords, and parameter names should be written with the GHATS macros:

```
\routine{gh_cs}
\keyword{help}
\param{power_err}
```

The argument should contain the literal name, with plain underscores. Do not write escaped underscores inside these macros. This keeps the source readable and avoids printed names such as `gh\_cs` in the PDF.

C.3 What Was Preserved

The LaTeX manual preserves the structure and tone of the legacy manual where possible: an introduction and installation section, a practical walkthrough style, command-oriented

chapters, and a compact reference appendix. The front matter also keeps the original spirit of the manual, including the revision history and the acknowledgement of Tomaso Belloni's authorship.

C.4 What Was Added

The new source adds material for functionality that post-dates parts of the legacy manual, including cross-spectral routines, dynamic-PDS FITS products, XSPEC export details, and the native GHATS file-format appendix. These additions should remain descriptive: they document what the current routines do and should not silently redefine the underlying Fourier, lag, normalization, or file-format conventions.

C.5 Maintenance Notes

When the manual is updated, the following checks are useful:

- keep examples runnable with the current GHATS calling sequences;
- update the routine directory and `gh_help` text when command behaviour changes;
- keep Appendix B synchronized with the Markdown file format specification or decide explicitly that the LaTeX appendix has become the primary source.